핵심만 간단히

https://wikidocs.net/book/1657

00_들어가기 전

숙련된 개발자가 새로운 언어를 가장 빨리 배우는 방법
모든 프로그래밍 언어는 변수, 연산자, 반복문, 예외 처리 등 몇 가지 개념으로 구성된다.
그러니 한 가지 프로그래밍 언어에 익숙하다면 다른 언어를 배우는 데 그리 어렵지 않을 수 있다.
어떤 한 언어에 익숙한 개발자가 다른 언어를 가장 쉽게 배우는 방법은 위와 같은 공통된 개념에 대해 차례대로 살피고 운동하듯이 몸이 익숙해질 때까지 그것을 사용하는 것이다. 그 과정의 고민과 실패는 성장의 동력이 될 것이다.
그때까지 이 책은 당신과 함께할 것이다.
이 책은 장황한 설명을 늘어놓지 않고 파이썬 핵심을 설명한다. 그리고 파이썬을 공부하면 반드시 부딪히는 고민을 다양한 코드와 예시를 통해 설명한다.
작은 책이지만 이 책이 다루는 주제는 넓고 깊다. 파이썬으로 데이터 분석 시 필요한 numpy, pandas, matplotlib 3종 패키지부터 최근 가장 뜨거운 반응을 보이는 '비동기식 프로그래밍', 메타 클래스 등 넓고 깊게 주제를 다룬다.
이제 여정을 시작하자.
저자 나익희

마지막 편집일시 : 2018년 11월 27일 4:27

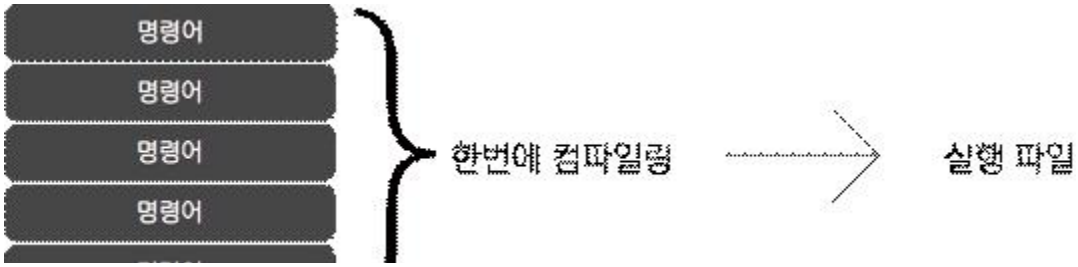
---

1. 02_인터프리터 언어란?

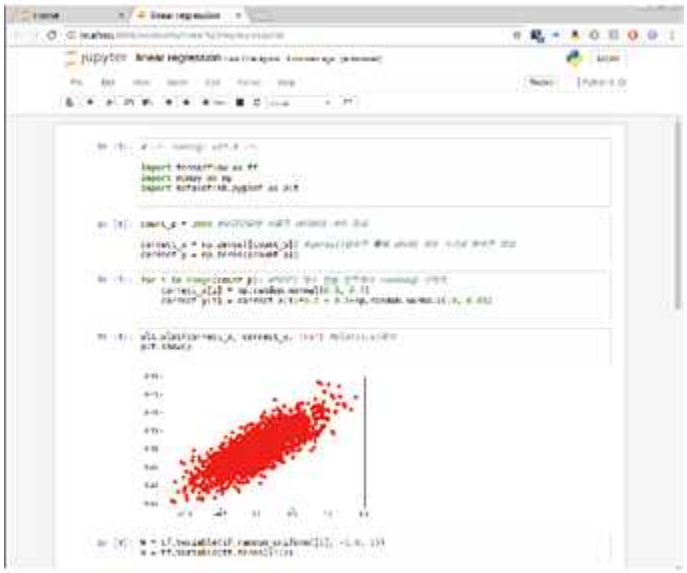
파이썬은 인터프리터 언어이다.

프로그래밍 언어는 **컴파일링 언어와 인터프리터 언어**로 나눌 수도 있다. 아래 그림은 그 차이점을 잘 나타낸다.

인터프리터 언어는 명령어 한 줄씩 컴파일링 되기 때문에 명령어 한 줄 한 줄이 중요할 수 있다. 그래서 Markdown 형식의 Jupiter 같은 것이 개발되었다.



컴파일링 언어는 한 번에 컴파일링 하므로 "구조"가 중요하다. 그래서 "객체지향성 언어"가 발전되었다.



위와 같은 화면을 Jupyter Notebook 이라 부르는데 그 이유는 마치 노트북에 필기하면서 수학 문제를 풀 듯 명령어 한 줄 혹은 여러 줄을 입력하면서 그 결과를 확인하고 다음 단계로 넘어가는 개발방식이기 때문이다. 이런 과정이 결국 알고리즘이 될 수 있으므로 개발이라고 부를 만하다.

인터프리터는 한 줄 명령어 단위이므로 명령 한 줄이 <함수>인 경우가 많다. 그래서 인터프리터 언어를 "함수 언어"라 부르기도 한다.

큰 틀에서 Linux 의 bash 쉘이나 zsh 쉘 그리고 Windows 의 cmd 도 "인터프리터 언어"라 부르기도 한다.

명령어 한 줄이 때론 너무 단순해 보이기도 하지만 그 안의 구조는 아주 복합할 수도 있고 기능에 필요한 모든 것이 담겨 있을 수 있다. 인터프리터 언어가 아주 단순한 한 줄 명령어라 하더라도 객체지향성 언어만큼 체계적으로 개발될 수 있다. R 혹은 파이썬도 이런 측면에서 아주 많이 발전되었다.

마지막 편집일시 : 2018 년 7 월 22 일 11:33

_____

01_화면에 "Hello World" 나타내기(파이썬 설치하기)
화면에 "Hello World"를 나타내는 것은 단순한 일이 아니다. 언어를 설치하고 메카니즘을 위한 구성을 이해해야 하고 기본 문법을 이해해야 한다. 그러니 Hello World를 출력하는 것은 중요한 일이다. 먼저 파이썬 3.7.0부터 설치하자. 참고로 Linux, OSX에는 파이썬 2.x이 기본적으로 이미 설치되어 있다. 우리는 3.x을 다룰 것이기 때문에 설치하는 것이다.

linux인 경우
터미널을 켜고 repo를 추가한다.
user$> sudo add-apt-repository ppa:jonathonf/python-3.6

apt-get를 최신으로 업데이트한다.
user$> sudo apt-get update

파이썬 3.7.0을 설치한다.
user$> sudo apt-get install python3.7

설치가 완료되었다. 이제 실행하고 "Hello World"를 찍어보자.

```
user$> python3.7
Python 3.7.0 (default, June  27 2018, 08:44:35)
[GCC 5.4.0 20160609] on linux
Type "help", "copyright", "credits" or "license" for more
information.
>>> print("Hello World!")
Hello World!
```

python 2.7은 이미 설치되어 있으므로
아래와 같이 명령하면 2.7이 실행되고
>>> python
아래와 같이 하면 3.7.0이 실행된다.
>>> python3.7

OSX인 경우
brew를 사용한다면 2줄이면 설치할 수 있지만, 가끔 OSX 업데이트로 인해 에러가 발생할 수 있다. 대부분 해결할 수 있지만, 지금은 번거롭더라도 홈페이지에서 내려받아 설치해보자. https://goo.gl/5MW5io

설치 후 다음과 같이 터미널에 명령어를 입력한다.

```
user$> python3
Python 3.7.0 (default, June 27 2018, 11:03:47)
```

```
[GCC 4.2.1 Compatible Apple LLVM 9.0.0 on darwin
Type "help", "copyright", "credits" or "license" for more
 information.

>>> print("Hello World!")
Hello World!
```

Hello World가 제대로 출력된다. 만약 brew를 사용한다면 아래와 같이 하면 된다.

```
user$> brew update
user$> brew install python3
user$> python3
Python 3.7.0 (default, June 27 2018, 11:03:47)
[GCC 4.2.1 Compatible Apple LLVM 9.0.0 on darwin
Type "help", "copyright", "credits" or "license" for more
information.

>>> print("Hello World!")
Hello World!
```

brew 설치 방법과 사용 방법은 따로 설명하지 않지만 아주 유용한 프로그램이니 사용하길 권한다.

Windows인 경우

아래 주소에서 내려받을 수 있다.

https://www.python.org/ftp/python/3.7.0/python-3.7.0.exe

"Add 파이썬 3.7 to PATH"를 체크 하고 설치한다. 어느 경로에서든 파이썬을 실행하기 위함이다. 설치 완료 후 cmd 혹은 powershell을 실행한다

컴맨드 창에서 다음과 같이 실행한다.

```
C:\Users\peter> python
Python 3.7.0 (v3.7.0:2c5fed8, June  27 2018, 18:11:49)
[MSC v.1900 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more
information.

>>> print("helloworld")
helloworld
```

Python에서의 "HelloWorld"의 의미
"Hello World"가 싱겁게 끝난 느낌이 있다. 그런 이유는 인터프리터 언어가 가진 특성 때문이다. MVC, OOP 등 전체를 컴파일링 해야 하는 언어는 구조를 파악하고 구성을 해야 해서 "Hello World"이 조금 더 복잡하다. 반면, 인터프리터 언어는 한 줄의 명령이 최소 단위이다. 앞으로 우린 한 줄 명령어들을 살펴볼 것이고 그것들의 의미를 파악할 것이다. 이런 한 줄 명령어 들이 모여 새로운 결과를 만들어 내고 서로 조화를 이루는 과정을 살펴볼 것이다.

마지막 편집일시 : 2018년 7월 22일 11:31

02_시작

이해하기

01_변수와 객체
파이썬에선 데이터형을 명시하지 않아도 된다. 기본적으로 다음과 같이 단순하게 정의한다.

```
>>> a = 3     #정수형 변수 선언
>>> a = 1.3   #실수형 변수 선언
>>> a = "안녕"     #문자형 변수 선언
>>> a = True     #논리형 변수 선언
>>> a = False
```

자료형은 데이터의 형태에 따라 자동으로 설정된다. 그리고 다음과 같은 명령으로 자료형을 확인 할수 있다.

```
>>> type(a)
<class 'int'>
```

정수란 의미로 int라고 알려준다. 근데 뭔가 특이하다. 자료형을 물었는데 앞에 class란 말이 붙어 있다. 왜, Class가 여기서 튀어나온단 말인가! 이것은 파이썬의 특징 중 하나인데 파이썬의 모든 변수는 객체이다. 파이썬을 깊게 들어가면 들어갈수록 파이썬이 얼마나 객체에 집착하는지 알게 될 것이다. 위 코드에서 만든 변수 a는 int라는 Class로 만들어진 객체이다.

```
>>> type(1.3)
<class 'float'>
>>> type("안녕")
<class 'str'>
>>> type(True)
<class 'bool'>
```

모두 각자의 자료형에 맞는 Class로 만들어졌다. int, float, str, bool은 builtin으로 설정된 Class들이다.

```
>>> int
<class 'int'>
```

int는 정확히 말하면 객체를 만드는 Class라고 말 할 수 있다. 하지만 파이썬을 스크립트 언어 혹은 함수형 언어이기도 하니 int를 그저 "함수"라고 불러도 큰 무리는 없을 것이다. 파이썬은 단순함을 유지하기 위해 만들어진 언어이다. 그래서 객체를 만드는 명령어 형식이 따로 있지 않고 함수를 실행하는 형식과 같다. 엄격함을 따지기보다는 유연하게 생각해 int(3.0)을 그냥 int()함수가 실행되었다고 말하든 객체를 만들었다고 말하든 캐스팅되었다고 말하든 틀리지 않을 것이다. 변수의 주소를 알고 싶으면 id() 함수를 사용한다.

```
>>> a = 5
>>> id(a)
1977114976
```

파이썬은 메모리 주소를 알려주는 함수는 있으나 포인터는 사용할 수 없다. 일찌감치 포인터에 대한 마음은 접는 것이 좋다. 정리해보자. 1. 변수 선언 시 자료형 명시하지 않는다. 2. 모든 변수는 객체이다. 3. 포인터는 없다.

more
파이썬 정수의 long타입은 없다. 파이썬3으로 넘어오면서 없어졌다. 정수는 그냥 정수이다.

실수형은 다음과 같이 표현도 가능하다.

```
>>> a = 1.3e1
>>> a
13.0
```

여러 개 변수를 동시에 선언할 수도 있다.

```
>>> a,b = 1,2
```

그리고 변수 삭제는 del이라는 예약어를 사용한다.

```
>>> a = 3
>>> del a
```

혹은

```
>>> a = 3
>>> del(a)
```

객체의 메서드 보기
파이썬의 모든 변수는 객체이다. 객체이니 당연히 메서드도 있다. 다음과 같이 확인할 수 있다.

```
>>> a = 3
>>> dir(a)
['__abs__', '__add__', '__and__', '__bool__', '__ceil__', '__
...이하 생략...
```

여러 목록이 나오는데 정수와 관련된 메서드이다. 문자열에는 어떤 메서드가 있는지 살펴보자.

```
>>> a = 'hello'
>>> dir(a)
['__add__', '__class__', '__contains__', '__delattr__', '__dir__',
'__doc__', '__eq__', '__format__', '__ge__', '__getattribute__',
'__getitem__', '__getnewargs__', '__gt__', '__hash__', '__
... 이하 생략...
```

마지막 편집일시 : 2018년 7월 22일 11:06

02_Reference 문서, 도움말 보기
파이썬에는 수많은 패키지와 모듈 그리고 함수가 있다. 많은 함수의 사용법을 알 수 없다. 그러니 도움말의 도움을 적절히 받는 것이 좋다.

함수에 대해 도움말 보기
```
>>> help(print)
```
위와 같이 하면 print() 함수에 대한 도움말을 볼 수 있다. 참고로 q를 누르면 도움말에서 빠져나올 수 있다.

특정 클래스의 메서드에 대해 도움말 보기
```
>>> help(str.split)
```
위와 같이 하면 str Class 안에 있는 split() 메서드에 대한 도움말을 볼 수 있다.

Python 공식 reference 문서
https://docs.python.org/3/reference/index.html

마지막 편집일시 : 2018년 7월 22일 11:06

03_리스트

리스트를 정의할 땐 중괄호 []를 사용한다. 다음과 같이 정의한다.

```
>>> a = [1, 2]
>>> b = ['apple', 'banana']
```

가장 큰 특징은 모든 요소는 서로 같은 형식이지 않아도 된다는 점이다.

```
>>> c = [1, 2, 'apple', 'banana']
```

리스트 안에 리스트를 다시 넣을 수도 있다.

```
>>> d = [1, 2, ['apple', 'banana']]      #리스트안에 리스트를 다시 넣을 수 있음
```

심지어 다른 형식의 벡터를 넣을 수도 있다.

```
>>> e = [1, 2, ('apple', 'banana')]        #()는 튜플을 의미한다.
```

괄호()를 사용하면 값을 변경할 수 없는 튜플인데 이 값도 리스트 에 넣을 수 있다.

빈 리스트 변수를 정의할 때는 아래와 같이 한다.

```
>>> f = [] #빈 리스트 만들기
```

인덱싱

index는 0부터 시작한다.

```
>>> a = ['apple', 'banana', 'orange', ('s', 'a')]
>>> a[0]
'apple'
```

리스트 안에 리스트가 들어가면 2차 구조가 되는데 이 때 다음과 같이 인덱싱 할 수 있다.

```
>>> a[3][0] #2차 구조 인덱싱
's'
```

'부분 선택'도 할 수 있는데 이때 : 기호를 사용한다.

```
>>> a = ['apple', 'banana', 'orange']
>>> a[1:3]
['banana', 'orange']
```

3이 아니라 2가 되어야 한다고 생각할 수도 있다. 다음 그림 을 보면 이유를 알 수 있다.

index는 요소 자체가 아니라 **자리를 의미한다.** 즉, 1:3이라고 하면 위 그림에서처럼 1부터3까지에 있는 요소가 선택 되는 것이다.

more

Comprehension 방식 사용하기

파이썬의 리스트를 다루는 좋은 방법의 하나는 Comprehension 방식을 사용하는 것이다. 다음과 같은 형태이다.

```
>>> a = [x*2 for x in range(6)]
>>> a
[0, 2, 4, 6, 8, 10]
```

0부터 시작해 여섯 개의 짝수를 만드는 명령이다. 위 명령은 두 부분으로 나눌 수 있다. for x in range(6)은 반복문이다. range(6) 함수에 의해 정수 0부터 5까지 값이 지정되고 이 값이 차례대로 선택된다.

x*2, 선택된 요소에 2를 곱했다. 즉, 짝수가 만들어진다. 즉, 위 명령을 말로 풀면 "0부터 6까지 순열을 만들고 요소 하나씩 꺼내 2를 곱한 결과를 리스트에 차례대로 담아라"이다.

comprehension은 숫자뿐만 아니라 문자열도 담을 수 있다.

```
>>> a = ['apple', 'banana', 'orange']
>>> [x for x in a if 'na' in x]
'banana'
```

위 명령은 a 리스트 요소 중 'na'문자열이 들어있는 요소만 선택하는 명령이다. 가장 먼저 for x in a가 눈에 띈다. a 리스트의 요소가 하나씩 선택된다. 그리고 조건문에 의해 'na' 문자열이 포함되어 있는지 in 예약어를 통해 조사한다. 그 후 만족하는 값을 리스트에 담게 된다.

이 외에도 comprehension은 다양한 방법으로 사용할 수 있다.

zip() 함수 사용하기

다음과 같은 두 개의 리스트가 있다.

```
>>> a = [1,2,3]
>>> b = [4,5,6]
```

[1,4]를 묶고 [2,5]를 묶고 [3,6]를 묶어야 할 때가 있다. 이때 사용하는 함수가 zip() 함수이다. 위와 같은 상황은 생각보다 꽤 많다. 이럴 때마다 복잡한 여러 줄의 코드를 작성하지 말고 다음과 같이 하면 된다.

```
>>> d = zip(a,b)
>>> list(d)
[(1, 4), (2, 5), (3, 6)]
```

근데 list(d)를 한 번 더 실행하면 빈 값을 반환한다. 당황스럽겠지만 iterator이기 때문인데 여기서 깊게 들어가면 너무 깊게 들어가니 그냥 기억해두자.

그래서 보통은 다음과 같이 변수에 저장해 둔다.

```
>>> a = [1,2,3]
>>> b = [4,5,6]
>>> e = list(zip(a,b))
>>> e
[(1, 4), (2, 5), (3, 6)]
```

마지막 편집일시 : 2018년 11월 20일 10:14

04_튜플(tuple)

튜플은 ()괄호를 사용해 정의한다. **리스트와 차이점은 튜플은 값을 변경할 수 없다는 것이다.**

```
>>> a = (1,2)    #괄호를 사용해 정의 내림
>>> del a[0]     #오류 발생
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'tuple' object doesn't support item deletion
```

값을 지우려 하니 에러가 발생했다.

```
>>> a[0] = 1     #수정할 수 없음.
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'tuple' object does not support item assignment
```

값을 수정하려고 하니 에러가 또 발생했다. 튜플은 괄호를 사용하지 않고 정의 할 수 있다.

```
>>> a = 1,2 #괄호 없어도 튜플을 만들 수 있음. 튜플 안에 튜플을 다시 넣을 수도 있다.
>>> a = ('a', 'b', ('c', 'd')) #튜플 안에 다시 튜플을 넣을 수 있음
```

또한, 리스트를 넣을 수도 있다.

```
>>> a = (1,2,[3,4])
```

이렇게 리스트를 넣으면 이 리스트 요소는 수정된다.

```
>>> a[2][0] = 6
>>> a
(1, 2, [6, 4])
```

비어 있는 튜플을 만들 때는 다음과 같이 한다.

```
>>> a = ()  #빈 튜플 만들기
```

주의할 점은 값을 한 개만 넣는다면 반드시 콤마를 넣어줘야 한다는 것이다.

```
>>> a = (1,)     #한 개일 때는 반드시 뒤에 ,를 붙여준다.
```

인덱싱

```
>>> a = (1, 2, 'c')
>>> a[1]
2
>>> a[1:3]
2. 'c'
```

리스트와 같은 형식으로 '부분 선택'이 된다.

```
>>> a = ('a', 'b', ('c', 'd'))
>>> a[1]
'b'
>>> a[2][0] #2차 구조 인덱싱
'c'
```

마지막 편집일시 : 2018년 7월 22일 11:10

05_딕셔너리(dictionary)

리스트과 비슷하지만 '요소'와 'key 값'이 매치되어 있다는 점이 다르다.

기본

```
>>> a = {'name' : 'hong', 'age' : 24}
>>> a
{'name': 'hong', 'age': 24}
>>> a['name']
'hong'
```

요소 추가하기

```
>>> a['height'] = 189
>>> a
{'name': 'hong', 'age': 24, 'height' : 189}
```

요소 삭제하기

```
>>> del a['height']
>>> a
{'name': 'hong', 'age': 24}
```

Key 값만 가져오기

```
>>> a = {'name' : 'hong', 'age' : 24}
>>> a.keys()
dict_keys(['name', 'age'])
```

value 값만 가져오기

```
>>> a = {'name' : 'hong', 'age' : 24}
>>> a.values()
dict_values(['hong', 24])
```

key, value 값 쌍으로 가져오기

```
>>> a.items()
dict_items([('name', 'hong'), ('age', 24)])
```

목록 모두 지우기
a.clear()
```
>>> a
{}
```

more
딕셔너리도 comprehension 방식으로 만들 수 있다. 아래 예시는 두 리스트를 하나의 딕셔너리 변수로 만드는 코드이다.

```
>>> a = ['apple', 'banana', 'orange']
>>> b = [7, 5, 8]
>>> d = {x : y for x, y in zip(a, b)}
>>> d
{'apple': 7, 'banana': 5, 'orange': 8}
```
zip() 함수로 묶은 뒤 딕셔너리 형식으로 저장했다.

마지막 편집일시 : 2018년 7월 22일 11:12

07_형변환(Casting)
다음과 같이 데이터를 다른 자료형으로 Casting 할 수 있다.

```
>>> float(3)     #실수형으로 바꿈
3.0

>>> int(3.0)     #정수형으로 바꿈
3

>>> str(3)  #문자열로 바꿈
'3'
```
다른 진수로 바꿀 수도 있다.

```
>>> hex(12) #16진수로 바꿈
'0xa'
>>> oct(10) #8진수로 바꿈
'0o12'
>>> bin(10) #2진수로 바꿈
'0b1010'
```
다시 10진수로 바꿀 때는 자동으로 바뀐다.

```
>>> a = 0b0010
>>> a
2
```

다른 진수값이 문자열이면 int() 함수를 사용한다.
```
>>> int('0xc', 16)
12
>>> int('0o14', 8)
12
>>> int('0b1100', 2)
12
```
두 번째 인자에 입력 값의 진수를 명시하면 된다. 다른 벡터 형으로 변환할 수도 있다.

```
>>> tuple([1,2])     #리스트를 튜플로 바꿈
(1, 2)

>>> list((1,2))  #튜플을 리스트로 바꿈
[1, 2]

>>> set([1,2])  #리스트를 집합 데이터형으로 바꿈
{1, 2}
```

딕셔너리로 캐스팅하는 것은 조금 까다로운데 key 값을 어떻게 설정할 것인가를 정해야 하기 때문이다. key와 value가 한 쌍으로 된 경우 딕셔너리로 캐스팅 할 수 있다.

```
>>> dict([[1,2],[3,4]])   #쌍으로 된 경우만 딕셔너리형으로 변환된다.
{1: 2, 3: 4}
```

유니코드로 변환은 다음과 같이 한다.

```
>>> ord('가')     #문자를 유니코드 값으로 변환
44032   #44032는 '가'의 유니코드 값
>>> chr(44032). #chr()함수는 유니코드 값을 문자로 변환한다.
'가'
```

마지막 편집일시 : 2018년 7월 22일 11:17

08_전역, 지역 변수 목록 보기

global 변수 보기

globals() 함수로 전역 변수를 확인할 수 있고 locals() 함수로 지역 변수를 확인할 수 있다.

두 변수를 만들어보자.

```
>>> a = 1
>>> b = 2
```

변수를 확인해 보자.

```
>>> globals()
{'__name__': '__main__', '__doc__': None, '__package__':
None, '__loader__': <class '_frozen_importlib.
BuiltinImporter'>, '__spec__': None, '__annotations__': {}, '__
builtins__': <module 'builtins' (built-in)>, 'a': 1, 'b': 2}
```

방금 만든 변수 a와 b가 맨 끝에 있다. __로 시작하는 변수가 많은데 이는 시스템에서 생성한 변수이다.

global 변수 local에서 사용하기

다음과 같은 코드가 있다.

```
>>> a = [1,2,3]
>>> def temp():
... a = [4,5,6]
```

전역 변수 a를 만들고 함수 내부에서 값을 변경했다.

```
>>> temp()
>>> print(a)
[1,2,3]
```

하지만, 값이 전혀 변하지 않았다. 이유는 temp()함수에서 접근한 a는 global 변수가 아니라 local 변수이기 때문이다. global 변수인 a에 접근하려면 a가 global 변수임을 알려줘야 한다.

```
>>> a = [1,2,3]
>>> def temp():
... global a
... a = [4,5,6]
```
global a를 추가해 a 변수가 전역 변수임을 알렸다.

```
>>> temp()
>>> print(a)
[4,5,6]
```
이제 제대로 반영되었다.

local 변수 보기
```
>>> def temp():
...     a= 3
...     print(locals())
>>> temp()
{'a': 3}
```
지역 변수 목록과 값이 나타난다.

마지막 편집일시 : 2018년 7월 22일 11:21

09_pyenv로 환경 만들기
파이썬 2.7.0, 파이썬 3.7.0 둘 다 사용해야 하는 경우가 있다.

Linux(Ubuntu) 혹은 OSX인 경우, 2.7.0은 이미 기본으로 설치되어 있고 파이썬 3.7.0은 설치하면 python3라는 명령어로 실행 가능하니 두 버전을 사용하는 것은 어려운 일도 아니다.

하지만 좀 더 전문적이고 체계적으로 파이썬을 관리할 필요가 있다. 방법 중 추천하는 방법은 pyenv 프로그램을 사용하는 것이다.

pyenv를 사용하면 버전별로 여러 환경을 만들어 사용 할 수 있다.

예를 들어 apple이라는 프로젝트를 시작했다면 python 2.7로 apple이라는 환경을 만든다. 이렇게 되면 이 환경에 어떤 패키지를 설치하더라도 별도 독립된 환경이기 때문에 다른 곳에 전혀 영향을 미치지 않는다. 만약 이 환경이 불필요하다면 삭제해 버리면 된다.

설치를 시작하자.

OSX인 경우
brew를 이용해 설치 할 수 있다.

```
user$> brew install pyenv
user$> brew install pyenv-virtualenv
```
.bash_profile파일을 연다.

```
user$> open ~/.bash_profile
```
아래 내용을 .bash_profile파일에 추가한다.

```
export PYENV_ROOT="$HOME/.pyenv"
export PATH="$PYENV_ROOT/bin:$PATH"
eval "$(pyenv init -)"
eval "$(pyenv virtualenv-init -)"
```

추가 후 터미널을 재시작한다.

Linux(Ubuntu)인 경우
apt-get으로 쉽게 설치할 수 없다. 아래와 같이 한다. 아래 내용을 타이핑 하기 어려우면 다음 페이지를 참고하자.

https://github.com/pyenv/pyenv/wiki/Common-build-problems

user$> sudo apt-get install -y make build-essential libssl-dev zlib1g-dev libbz2-dev libreadline-
dev libsqlite3-dev wget curl llvm libncurses5-dev libncursesw5-dev xz-utils tk-dev

user$> curl -L https://raw.githubusercontent.com/
pyenv/pyenv-installer/master/bin/pyenv-installer | bash
설치 후 .bashrc 파일에 다음 내용을 추가해야 한다.

user$> gedit ~/.profile
export PATH="~/.pyenv/bin:$PATH"
eval "$(pyenv init -)"
eval "$(pyenv virtualenv-init -)"
추가 후 shell을 재시작한다.

Windows인 경우
안타깝지만 공식적으로 Windows를 지원하지 않는다.

설치 가능한 파이썬 목록 보기
user$> pyenv install —list
설치할 수 있는 파이썬 버젼이 나온다. 이 중에서 원하는 버젼을 다음과 같이 쉽게 설치할 수 있다.

Python 설치하기
user$> pyenv install 2.7.13
user$> pyenv install 3.7.0a3
2.7.13과 3.7.0을 각각 설치했다. 두 버젼은 독립된 환경이다. 2.7.13을 사용하고 싶다면 다음과 같이 할 수 있다.

Python 버젼 사용하기
user$> pyenv shell 2.7.13
이 상태에서 python이라고 명령하면 2.7.13이 실행된다. 만약 pyenv shell 3.7.0a3이라고 명령하고 python을 실행하면 3.7.0이 실행된다. 원래 system의 python으로 돌아오고 싶다면 pyenv shell system이라고 명령하면 된다.

사용 가능한 Python 목록을 보고 싶다면
user$> pyenv versions
위와 같은 명령을 내리면 현재 설치된 python 버젼과 만들어진 환경목록을 보여준다.

그럼 이제 독립된 이름을 가진 환경을 만들어보자.

환경 만들기
user$> pyenv virtualenv 2.7.13 gamesu
gamesu이라는 환경을 파이썬 버젼 2.7.13으로 만들었다. gamesu는 다른 버젼과 환경으로부터 완전히 독립된 공간이다.

만들어진 gamesu환경을 사용하려면 다음과 같이 한다.

만들어진 환경을 사용하기
user$> pyenv activate gamesu
혹은

user$> pyenv shell gamesu
activate는 이름을 가진 환경에만 적용할 수 있다. 반면 shell은 만들어진 환경뿐만 아니라 특정 버젼을 이용할 때도 사용할 수 있다.

user$> pyenv shell gamesu
user$> pyenv shell 3.7.0a3
user$> pyenv activate 3.7.0 (에러 발생함)

환경에서 빠져나오기
activate로 실행했다면
user$> pyenv deactiavte

shell로 실행했다면
user$> pyenv shell system

환경 지우기
user$> pyenv **uninstall** gamesu
마지막 편집일시 : 2018년 7월 22일 11:26

10_주석달기(Comment)
Pycharm과 같은 IDE에서 파이썬을 사용한다면 다음과 같은 방법으로 주석 처리할 수 있다.

한 줄
```
#    print("Hello World")
```

여러 줄
```
"""
print("Hello World")
print("Hello World")
"""
```
마지막 편집일시 : 2018년 7월 22일 11:28

11_Console로 출력하기
파이썬은 코드가 터미널에서 실행되는지 IDE 실행되는지 알고 있다. 코드가 터미널에서 실행되고 있다면 변수명만 입력해도 값을 화면에 보여준다.

```
# 터미널에서 실행한 경우
>>> a = 3
>>> a
#결과
3
```

하지만 IDE에선 변수명만으로 값을 보여주지 않는다.
```
# IDE에서 실행한 경우
a = 3
a
#결과
```

print() 함수를 사용해야만 한다.

```
a = 3
print(a)
#결과
3
```

코딩 시 값을 확인하기 위해 Console에 출력하는 경우는 많다. 다음과 같이 확인한다.

여러 사용 방법을 살펴보자.

단일 값

```
>>> print('안녕')
안녕
```

여러 값

```
>>> print('정말', '안녕')
정말 안녕
>>> print('정말'+'안녕')
정말안녕
>>> print('정말''안녕')
정말안녕
```

,를 사용하면 띄워 쓰고 +혹은 ' 기호를 사용하면 붙여 출력된다.

```
>>> >>> print('정말', '안녕', end="***")
정말 안녕***>>>
```

print()는 원래 맨 마지막에 \n을 추가해 줄 바꿈을 한다. 그런데 end인자를 추가하면 원하는 값으로 출력할 수 있다. 위의 경우 마지막에 \n 대신 ***이 출력되었다. 그래서 파이썬 프롬프트가 줄 바꿈이 되지 않았다.

```
>>> a = "hello world"
>>> for i in a:
... print(i, end=" ")
h e l l o   w o r l d
```

위 예시 경우, **원래 end 인자가 없이 print(i)하면 한 글자씩 줄 바꿈 해서 나오는데 end인자에 공백이 설정되어 줄 바꿈 대신 공백이 붙어 출력되었다.**

포맷 형식으로 출력하기

```
>>> aa = 30
>>> print("%s은 정수입니다"%aa)
30은 정수입니다
```

줄 바꿈

```
>>> print("Hello\nWorld")
Hello
World
```

more

복잡한 딕셔너리 변수면 한 줄로 이어 나오기 때문에 불편하다.

```
>>> obv = dict(name='peter', age=24, score=85)
>>> print(obv)
{'score': 85, 'age': 24, 'name': 'peter'}
```

보기 불편한 정도는 아니지만 좀 더 보기 좋게 출력할 수 있다.

```
>>> from pprint import *
>>> pprint.pprint(obv, width=20, indent=4)
{    'age': 24,
    'name': 'peter',
    'score': 85}
```

pprint라는 외부 모듈을 사용해 값을 보기 좋게 출력했다. from pprint import *는 모듈을 불러오는 방법이다. 뒤에서 자세히 다룬다.

마지막 편집일시 : 2018년 7월 24일 8:07

12_반올림, 내림, 올림

올림
math 모듈을 사용한다. math 모듈에는 많은 수학 함수가 있다.

```
>>> import math #math 모듈을 먼저 import해야 한다.
>>> math.ceil(-3.14)     #결과는 -3
>>> math.ceil(3.14)      #결과는 4
```

내림
이 역시 math 모듈을 사용한다.

```
>>> import math
>>> math.floor(3.14)     #결과는 3
>>> math.floor(-3.14)    #결과는 -4
```

내림에는 다른 함수가 하나 더 있다. math.trunc()함수이다. 차이점을 예시를 통해 살펴보자.

```
>>> math.trunc(-3.14)    #결과는 -3
>>> math.floor(-3.14)    #결과는 -4
```

trunc()함수는 내림을 하더라도 0으로 향하는 반면 floor() 함수는 무조건 아래만 향해 내림한다. 참고로 math.trunc()함수는 int()와 같이 결과를 반환한다.

```
>>> int(-3.14)             #결과는 -3
>>> math.trunc(-3.14)    #결과는 -3
```

반올림
파이썬에 내장된 round() 함수를 사용한다.

```
>>> round(3.1415)    #결과는 3
```

원래 두 개의 인자를 받는데 두 번째 인자가 생략되면 소수 첫째 자리에서 반올림한다. 두 번째 인자를 살펴보자.

```
>>> round(3.1415, 2)    #결과는 3.14
```

2를 넣었더니 소수 셋째 자리에서 반올림한다. 두 번째 인자에 음수를 사용할 수 있다.

```
>>> round(31.415, -1)    #결과는 30.0
```

사사오입 원칙
round()는 사사오입 원칙을 따른다. 반올림할 자리의 수가 5이면 반올림 할 때 앞자리의 숫자가 짝수면 내림하고 홀수면 올림 한다.

```
>>> round(4.5)  #결과는 4
>>> round(3.5)  #결과는 4
```

앞의 자리가 4인 경우 짝수니까 내림이 되었고 앞의 자리가 3인 경우 홀수니까 올림이 적용되었다.

마지막 편집일시 : 2018년 7월 24일 8:09

13_산술 연산자 Arithmetic Operators
기호에 의해 산술적인 연산을 할 수 있는 기호들이다. 크게 다른 언어와 다른 부분은 없다.

산술 연산자
```
>>> 4**2     #지수
16
>>> 5*2 #곱하기
10
>>> 5/2 #나누기
2.5
>>> 5//2     #몫 구하기
2
>>> 5%2 #나머지 구하기
1
>>> 5+2 #더하기
7
>>> 5-2 #빼기
3
```
마지막 편집일시 : 2018년 7월 24일 8:11

14_내장 산술 함수
내장된 산술과 관련된 함수를 살펴보자.

```
>>> sum([1,2,3,])
#결과는 6, 요소를 모두 더한다. 리스트값을 제공해야 한다.

>>> max([1,2,3])
#결과는 3, 요소 중 가장 큰 값을 반환한다.

>>> min([1,2,3])
#결과는 1, 요소 중 가장 작은 값을 반환한다.

>>> abs(-2) #결과는 2, 절댓값

>>> pow(4, 2)
#결과는 16, 4의 2제곱, pow(4,2)는 4**2와 같다. pow() 함수를
다음과 같이 사용할 수도 있다.

>>> pow(4,2,4)
#결과는 0, 위 명령은 4의 제곱을 다시 4로 나눈 나머지를
반환한다. 즉, 4**2 % 4와 같다.

>>> divmod(5,2)
#결과는 (2,1), 5를 2로 나눈 몫과 나머지를 반환한다.

>>> round(3.14)
#결과는 3, 반올림 함수이다.
```
마지막 편집일시 : 2018년 7월 24일 8:11

15_math 모듈(Math Module)
좀 더 복잡한 연산이 필요한 경우 math 모듈을 사용한다. 모두 기억하는 것보다 필요할 때 찾아보는 것이 좋다.

정수 관련 함수
>>> import math
우선 위와 같이 math 모듈부터 import 해야 한다.

>>> math.ceil(3.14) #4
위 함수는 '올림' 함수이다. 인자는 하나만 필요로 한다. 결국, 소수점을 올림 하여 정수로 만드는 것이다.

>>> math.copysign(3.14, -3) #-3.14
두 번째 인자의 부호만 취해 첫 번째 인자에 적용한다. **부호만 취하는 경우는 생각보다 많다.**

>>> math.fabs(-3.14) #3.14
절댓값을 반환한다.

>>> math.factorial(5) #120
팩토리얼 함수인데 팩토리얼은 1부터 인자로 주어진 값까지 모두 곱하는 값을 반환한다. 위의 경우 12345를 모두 곱한 것이다.

>>> math.floor(3.78) #3
내림 함수이다. 한 개 인자만 취하며 결국 소수점을 내려 정수로 만든다. 0에 가깝게 내리는 것이 아니라 그냥 작은 값으로 내린다.

>>> math.frexp(100)
(0.78125, 7)
입력받은 값이 m * 2**e와 같은 값을 가지는 m과 e를 반환한다. 예를 들어 100을 인자로 넣으면 (0.78125, 7)을 반환하는데 이를 다음과 같이 계산해보면 정확히 100이 나온다.
>>> 0.78125*2**7 #100.0

>>> math.ldexp(0.78125, 7) #100
math.frexp()의 반대인데 m2*e에 각각 대입되어 계산되어 값을 반환받는다. m이 첫 번째 인자이고 e가 두 번째 인자이다.

>>> math.**gcd(6, 8) #2**
두 수의 최대 공약수를 반환한다.

>>> math.modf(3.14) #(0.14000000000000012, 3.0)
math.modf() 함수는 입력값을 정수와 소수 부분으로 분리해 반환한다. 근데 위의 경우 소수가 0.14가 아니라 유달리 길다. 이는 부동소수점 문제이다. 부동소수점은 10진법 수를 2진법 체계에서 정확히 반영하지 못해 생기는 문제이다. modf()함수는 부동소수점의 값을 그대로 반환한다.

>>> math.trunc(3.14) #3
내림 함수이다. math.floor와 비슷해 보이지만 서로 다르다. 아래 예시를 보자.

>>> math.trunc(-3.14) #-3
>>> math.floor(-3.14) #-4
trunc()함수는 0으로 향하지만 floor()함수는 무조건 아래로만 향한다.

로그 함수와 제곱
>>> math.log(10, 10) #1

math.log(a, b)는 로그 함수이며 b를 밑으로 하는 log a에 대한 로그 값을 리턴한다.

>>> math.log1p(x)
e를 밑으로 하는 x+1로그

>>> math.log2(x)
2를 밑으로 하는 x로그

>>> math.log10()
10을 밑으로 하는 x로그

>>> math.pow(3, 2) #9
math.pow(x, y)는 x에 y 승을 계산한 결괏값을 반환한다.

>>> math.sqrt(25) ##결과는 5.0이다.
제곱근의 값을 반환한다.

삼각함수
>>> math.acos(x) #아크코사인
>>> math.asin(x) #아크사인
>>> math.atan(x) #아크탄젠트
>>> math.atan2(y, x) #y/x 아크탄젠트
>>> math.cos(x) #코사인
>>> math.sin(x) #사인
>>> math.tan(x) #탄젠트
각도 변환
>>> math.degrees(x)
라디안으로 표현된 각도를 60분법 각도로 변환

>>> math.radians(x)
60분법으로 표현된 각도를 라디안 각도로 변환

상수
>>> math.pi ##결과는 3.141592653589793이다. 원주율
>>> math.e ##결과는 2.718281828459045이다. 자연 상수
>>> math.tau ##결과는 6.283185307179586이다. 타우
마지막 편집일시 : 2018년 7월 24일 8:19

16_비트 연산자(Bitwise Operators)
비트 쉬프트
>>> 2<<1 #0010에서 0100으로 변경, #결과는 4
>>> 2<<2 #0010에서 1000으로 변경, #결과는 8
>>> 8>>2 #1000에서 0010으로 변경, #결과는 2
비트 연산
>>> 6^4 #0110 + 0100, xor 연산
2 #0010, 둘 중 한 쪽만 1이 있는 경우만 참
>>> 2|4 #0010 + 0100, or 연산
6 #0110, 둘 중 한 쪽에 1이 있기만 하면 참
>>> 2&6 #0010 + 0110, and 연산
2 #0010, 양 쪽 모두 1이 있는 경우 참
>>> ~2 #10, not 연산
-3 #-11
2진수로 변환

2진수로 변환은 bin() 함수를 사용한다.

```
>>> bin(10) #2진수로 바꿈
'0b1010'
```
다시 10진수로 바꿀 때는 자동으로 바뀐다.

```
>>> a = 0b0010
>>> a    ##결과는 2
```
자동으로 바뀌는 것을 방지하기 위해서는 문자열로 저장해야 한다. 그리고 문자열로 된 2진수를 다시 10진수로 바꾸기 위해서는 int() 함수를 사용한다.

```
>>> a = '0b0010'
>>> a
'0b0010'
>>> int(a, 2)
2
```
마지막 편집일시 : 2018년 11월 20일 10:28

17_비교 연산자(Comparison Operators)
```
>>> 5 <= 2
False
>>> 5 < 2
False
>>> 5 > 2
True
>>> 5 >= 2
True
>>> 5 == 5
True
>>> 5 != 5
False
```
마지막 편집일시 : 2018년 7월 24일 8:21

18_할당 연산자(Assignment Operators)
```
>>> a = 5
>>> a %= 5  #a = a % 5
>>> a /= 5  #a = a / 5
>>> a //= 5     #a = a // 5
>>> a -= 5  #a = a - 5
>>> a += 5  #a = a + 5
>>> a *= 5  #a = a * 5
>>> a **= 5     #a = a ** 5
```
마지막 편집일시 : 2018년 7월 24일 8:21

19_식별 연산자(Identity Operators)
```
>>> a = 5
>>> b = 5
>>> a is b
True
>>> a is not b
False
```

위 코드는 a와 b의 값이 5인가를 테스트 하는 것이 아니라 a와 b가 같은 메모리 주소를 가졌는가를 테스트하는 것이다.

다음 예를 보자.

```
>>> 5 == 5.0
True
>>> 5 is 5.0
False
```

== 연산자는 값을 비교하기 때문에 5와 5.0은 같다고 반환한다. 하지만 is연산자는 객체의 주소가 서로 달라서 False를 반환한다.

more
파이썬은 정수 중 -5부터 256까지는 이미 객체를 만들어 메모리에 둔다.

```
>>> a = 256
>>> b = 256
>>> a is b
True
```

하지만 -5부터 256 밖에 있는 정수는 저장되어 있지 않다.

```
>>> a = 257
>>> b = 257
>>> a is b
False
```

mutable vs immutable
두 가지 상황에 대해 살펴보자. b 변수에 a를 대입하고 주소를 비교하면 같은 주소를 사용한다.

```
>>> a = 257
>>> b = a
>>> a is b
True
```

a에 다른 값을 대입해보자.

```
>>> a = 258
>>> b
257
```

b는 변경되지 않는다. 정수는 immutable이다. 따라서 변수에 변수가 대입된 상태에서 한쪽이 바뀌면 서로 다른 메모리를 가리키게 된다.

하지만 리스트는 다르다.

```
>>> a = ['aa']
>>> b = a
>>> a is b
True
>>> a[0] = ['aaa']
>>> a
[['aaa']]
>>> b
[['aaa']]
>>> a is b
True
```

한쪽의 값이 바뀌면 다른 한쪽의 값도 바뀐다. 메모리 주소가 같다. 리스트는 mutable이기 때문이다. 그래서 참조하게 하지 않으려면 복사해줘야 한다.

```
>>> import copy
>>> b = copy.deepcopy(a)
```

이렇게 하면 b와 a는 완전히 다른 객체가 된다. 심지어 만약 a가 2차 구조를 가진 리스트였다 하더라도 그 하위까지 모두 copy 된다.

만약 2차 구조가 아닌 1차 구조의 단순한 변수라면 아래와 같이 간단히 복사할 수도 있다.

```
>>> b = a.copy()
```

마지막 편집일시 : 2018년 7월 24일 8:23

20_멤버 연산자(Membership Operators)

리스트에 적용할 경우

예약어 in은 특정 값이 리스트 혹은 문자열에 속해있는지 알려주는 연산자이다. 예제를 살펴보자.

```
>>> 1 in [1, 2]
True
```

1이 리스트 요소에 속해있으니 True 값을 반환한다. 다음 예제처럼 '속하지 않은 경우'를 검사할 수도 있다.

```
>>> 3 not in [1, 2]
True
```

문자열에 적용할 경우

문자열이 다른 문자열에 속하는지도 in 연산자를 통해 알 수 있다.

```
>>> ‘ffe’ in ‘coffee’
True
>>> ‘ffe’ not in ‘coffee’
False
```

마지막 편집일시 : 2018년 7월 24일 8:24

21_논리 연산자(Logical Operators)

대부분의 언어에서 &, |, !이 사용되지만 파이썬은 and, or, not 영어 단어 그대로 사용한다.

```
#not 연산자
>>> a = 5
>>> if not a == 4:
...     print("OK")
OK
#and 연산자
>>> a = 2
>>> b = 3
>>> if a==2 and b==3:
...     print("OK")
OK
#or 연산자
>>> a = 2
>>> b = 3
>>> if a==2 or a==5:
```

```
...     print("OK")
OK
```
마지막 편집일시 : 2018년 7월 24일 8:25

22_문자열 처리하기
"hello"라는 문자열은 ['h', 'e', 'l', 'l', 'o']처럼 요소로 이루어진 리스트처럼 여겨진다. 따라서 다음과 같이 인덱싱할 수 있다.

```
>>> a = "hello"
>>> a[0]
'h'
```
문자열 부분 선택(슬라이싱)
다음과 같이 부분 선택을 할 수 있다.

```
>>> a = "apple"
>>> a[1:3]
'pp'
```
index에 3이 들어갔으면 'ell'이 나와야 할 것 같은데 'el'이 나온 것이 좀 이상하다. 다음 그림을 보면 쉽게 이해할 수 있다.

index가 글자 자체가 아니라 위치이다. 숫자를 생략할 수도 있다.

```
>>> a[:4]
'appl'
>>> a[2:]
'ple'
```
첫 번째 index를 생략하면 맨 처음이 선택되고 두 번째 index를 생략하면 맨 끝이 선택된다.

for 문도 돌릴 수 있다.

```
>>> for s in a:
...     print(s)
h
e
l
l
o
```
리스트처럼 하나의 요소로써 문자가 취급된다.

문자열 합치기
```
>>> a = "hello"
>>> b = "world"
>>> a + b
"hello world"
```

연산자를 사용하면 문자열끼리 합칠 수 있다.

문자열 반복
>>> "안녕"*3
'안녕안녕안녕'

다른 언어에선 쉽게 볼 수 없는 방법이지만 python에선 문자열과 정수를 곱하기 연산할 수 있다. 다음과 같은 상황에 유용하다.

>>> "-"*50
——
문자열에 따옴표 표현하기
>>> a = '모두 "안녕"이라 인사해요'
>>> a = "모두 '안녕'이라 인사해요"
작은따옴표와 큰따옴표를 겹쳐서 사용하지 않으면 문자열 안에서 그대로 인식된다.

\" 이 기호는 Grep 정규 표현식인데 문자열 안에서 사용 할 수 있다.
>>> a = "모두\"안녕\"이라 인사해요"
>>> a
'모두"안녕"이라 인사해요'

more
검색
>>> a = 'hello'
>>> a.find('ll')
2
find() 함수로 문자를 검색하면 index 위치를 알려준다.

문자열 길이 조사 - len()
>>> a = 'hello'
>>> len(a)
5
len() 함수는 리스트의 요소 개수와 문자열의 글자 개수를 알려주는 함수이다.

문자열 바꾸기 - replace()
>>> a = "Good morning"
>>> a.replace("morning", "afternoon")
'Good afternoon'

문자열 나누기 - split()
>>> a = "Good morning"
>>> a.split()
['Good', 'morning']
split() 함수에 아무 인자도 없으면 공백을 기준으로 나누지만, 특정 문자를 사용하면 그 문자를 기준으로 나뉜다.

>>> a = "hello...world"
>>> a.split('...')
['hello', 'world']
만약 문자열을 한 글자씩 나눠 저장하고 싶다면 다음과 같이 간단히 list로 Casting하기만 하면 된다.

>>> a = 'hello'

```
>>> list(a)
['h', 'e', 'l', 'l', 'o']
```

대문자로 바꾸기 - upper()

```
>>> a = 'hello'
>>> a.upper()
HELLO
```

소문자로 바꾸기 - lower()

```
>>> a = 'HELLO'
>>> a.lower()
'hello'
```

공백 지우기 - strip()

```
>>> a = 'hello    '
>>> a.strip()
'hello'
```

왼쪽, 오른쪽 공백 모두 지워진다. 한쪽만 지우고 싶다면 다음과 같이 한다.

왼쪽 혹은 오른쪽 공백 지우기 - lstrip()

```
>>> a.lstrip()  #왼쪽 공백 지우기
>>> a.rstrip()  #오른쪽 공백 지우기
```

마지막 편집일시 : 2018년 7월 24일 8:30

==========

23_프롬프트로 사용자 문자 입력받기

다음과 같이 간단히 사용자 입력을 받아 변수에 저장할 수 있다.

```
>>> a = input("인사하세요 : ")
인사하세요 : hello world
>>> a
'hello world'
```

마지막 편집일시 : 2018년 7월 24일 8:30

---

2. 24_그래픽 개발 환경 만들기(Pycharm)

인터프리터 언어는 기본적으로 터미널에서 작동한다.

하지만 긴 코딩을 해야 할 때는 불편하다. 그래픽 개발 환경을 만들어보자. 여러 방법이 있지만, 필자는 Pycharm community을 추천한다.

파이썬만을 위해 만들어졌고 코딩 내용 중 해당 패키지가 없다면 이를 알려주고 설치까지 도와준다. 자동완성은 기본이다.

가. Pycharm community로 "Hello World" 화면에 나타내기

https://goo.gl/nF9Zw9

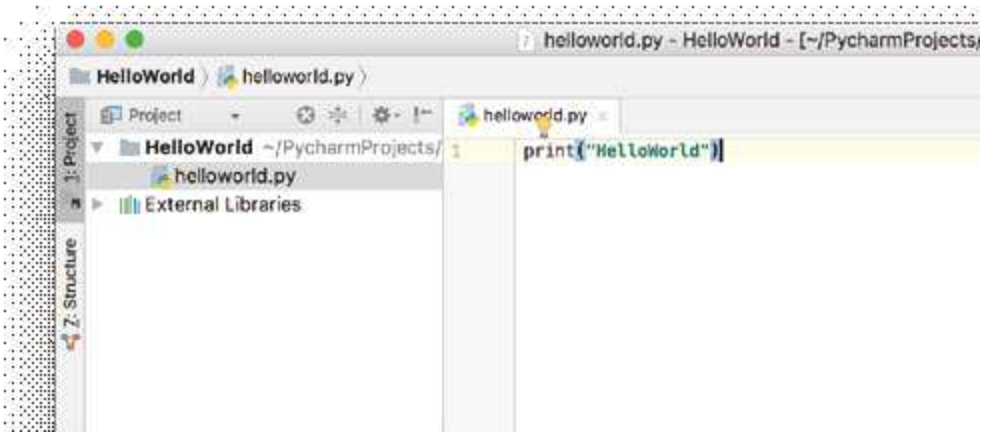
위 주소에서 내려받아 설치한다.

Create New Project를 클릭한다.

pyenv에서 환경을 만들었다면 Interpreter에 나타난다. 3.x관련된 Interpreter를 선택한다. 프로젝명은 "HelloWorld"로 한다.

프로젝트명에서 마우스 오른쪽 버튼을 눌러 New -> 파이썬 File을 선택한다.

코딩한다.



메뉴 중 Run -> Run을 선택한다.

마지막 편집일시 : 2018년 7월 24일 9:10

==========

25_function
파이썬에선 function도 객체이다. 따라서 함수를 변수에 저장할 수도 있고 리스트 요소로 넣을 수도 있고 다른 함수의 인자로 보낼 수도 있다. 하나씩 살펴보자.

먼저 함수를 정의하는 방법부터 살펴보자.

```
def add(x,y):
    return x + y
add(3,5)
#결과
8
```

파이썬은 function이라는 예약어를 사용하지 않고 def 예약어를 사용한다. {}도 없다. :기호를 사용하고 내용을 들여쓰기만 하면 된다.

함수는 다른 변수에 대입될 수 있다.

```
a = add
a(1,2)
#결과
3
```

대입한 변수를 통해 함수가 실행되었다.

함수를 리스트에 넣어 사용할 수도 있다.

```
l = [3, 4, add]
print(l[2](1,5))
#결과
6
```

함수를 다른 함수에 전달할 수도 있다.

```
def oh(func, x, y):
    return func(x, y)

print(oh(add, 3,5))
#결과
8
```

이쯤 되면 어디까지 되는지 확인하고 싶은 마음이 생길지도 모른다. 심지어 함수 안에 함수를 넣을 수도 있다.

```
def ah(x):
    def ah2(y):
        return x+y
    return ah2

ah(100)(2)
#결과
102
```

흔히 볼 수 없는 함수 실행이다. ah(100)으로 먼저 ah()함수를 호출하고 결과로 ah()함수 안에 있는 ah2()함수를 반환 받는다. 그리고 다시 2인자를 전달하여 호출하게 된다. 중요한 점은 ah(100)에서 이미 전달한 인자 100이 살아 있다는 사실이다.

한 줄 function 만들기

한 줄 function을 만들 수 있다.

```
>>> sum = lambda a, b: a+b
>>> sum(3,4)
7
```

more
args, *kwargs 인자에 대해
파이썬을 공부하다 보면 위와 같은 인자를 자주 보게 된다. function에 정해지지 않은 인자 개수를 전달하기 위해 인자 앞에 * 기호를 붙인다.

```
def temp(*args):
    print(args)
temp(1,2,3)
temp("hello", 5, 7, ["Good", "Morning"])
#결과
(1, 2, 3)
('hello', 5, 7, ['Good', 'Morning'])
```

temp()함수에 요소의 개수와 상관 없이 넣을 수 있고 어떤 데이터형이든 넣을 수 있다. temp()함수에서는 이 인자를 튜플로 처리한다. ** 기호를 사용하면 인자를 딕셔너리 형태로 전달 할 수 있다.

```
def temp2(**kwargs):
    print(kwargs)
temp2(a=3, b=4, c="hello")
#결과
{'a': 3, 'b': 4, 'c': 'hello'}
```

key와 value를 전달 했다. 함수에선 이를 딕셔너리로 처리한다. 와 *를 함께 사용 할 수도 있다.

```
def temp3(*arg, **kwargs):
    print(arg)
    print(kwargs)
temp3(6,7,8, a=1, b=2, c=3, d="hello")
#결과
(6, 7, 8)
{'a': 1, 'b': 2, 'c': 3, 'd': 'hello'}
```

튜플과 딕셔너리를 분리하여 별도로 처리 된다.

마지막 편집일시 : 2018년 7월 24일 8:35

26_if문
{} block을 사용하지 않는다. 들여쓰기만 잘 하면 된다.

```
a = 10
if a < 11:
    print("Hello")
#결과
Hello
```

몇 칸을 들여 쓰는 것이 좋은가?는 개발자마다 다르지만, **보통 탭 혹은 4칸을 들여 쓴다.**

else, else if

```
a = 10
if a < 8 :
    print("A")
elif a < 11 :    #else if 대신 elif를 사용한다.
    print("B")
else :
    print("C")

#결과
B
```

more

여러 조건 추가하기(and, or, not)

별도의 기호 없이 and, or, not 이 단어 그대로 사용하면 된다.

```
>>> a = 3
>>> if a > 2 and a < 4 :
...     print("OK")
#결과
OK
```

두 조건을 만족하므로 print()문이 실행되었다.

```
>>>if a>1 or a>4:
...     print("OK")
#결과
OK
```

두 조건 중 하나라도 만족해 print()문이 실행되었다.

```
>>> if not a==2:
...     print("OK")
#결과
OK
```

마지막 편집일시 : 2018년 7월 24일 8:36

27_for 문

순차적인 번호 사용

for 문에서 순차적인 번호를 사용하기 위해서는 1,2,3과 같이 순열이 있는 리스트 변수가 필요하다. range() 함수가 순열을 만들어 준다.

```
>>> list(range(10))
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> for i in range(10):
...     print(i)
i = 0
i=1
i=2
i=3
…생략…
```

요소를 사용
range()대신 리스트 변수를 사용하면 된다.

```
words = ["apple","orange","banana"]
for word in words:
    print(word)
#결과
apple
orange
banana
```

more
for 문의 리스트 요소가 튜플 혹은 리스트일 때 처리 방법
다음과 같이 2차 구조 리스트 변수인 경우가 있다.

```
words = [('apple','good'), ('banana', 'bad'),
                ('orange', 'excellent')]

for word in words:
    print(word)
#결과
('apple', 'good')
('banana', 'bad')
('orange', 'excellent')
```

이런 경우 다음과 같이 하면 된다.

```
for (word1, word2) in words:
    print(word1)
#결과
apple
banana
orange
```

for 문에 두 개의 변수를 설정해 자유롭게 사용할 수 있다.

break 예약어 사용하기

```
for i in range(10):
    print(i)
    if i > 5 : break
#결과
0
1
2
3
4
5
6
```

i가 6이 되면 if의 조건을 만족하기 때문에 break에 의해 for 문이 중단된다.

continue 예약어 사용하기

```
for i in range(10):
    print(i)
    if i > 5 : continue
    print("--------")
```

#결과
0

1

2

3

4

5

6
7
8
9
i의 값이 6이 되면 if의 조건을 만족하기 때문에 continue가 실행되어 더 이상 "--------"가 실행되지 않았다.

마지막 편집일시 : 2018년 7월 24일 8:38

28_while 문
while 문은 조건이 참일 때까지 반복한다.

```
i = 0
while i < 6:
    i += 1
    print(i)
else:
    print("end")
```
#결과
1
2
3
4
5
6
end
6보다 작을 때까지 반복 실행되었다.

break 예약어 사용하기
```
i = 0
while i < 5:
    i += 1
    if i == 3:
        break
    print(i)
```
#결과
1
2
i가 3이 되면 break가 실행되어 while 문이 종료된다.

continue 예약어 사용하기

```
i = 0
while i < 5:
i += 1
if i == 3:
    continue
print(i)
```

i가 3이 되면 continue 문이 실행되어 다음 아랫줄이 실행되지 않고 다음 스텝으로 넘어간다. 그래서 3이 빠지고 프린트되었다.

```
#결과
1
2
4
5
```

만약 무한루프 돌게 하고 싶다면 다음과 같이 한다.

```
while True:
    print("짠")
    print("단")
```

마지막 편집일시 : 2018년 7월 24일 8:40

29_외부 모듈 불러오기(Module)

다른 파이썬 파일을 불러와 사용할 수 있다. 불러온 파이썬 파일에 클래스가 정의되어 있다면 객체를 만들 수도 있고 그 파일에 단순히 함수들만 정의되어 있다면 함수들을 사용할 수 있다.

파이썬에선 이렇게 외부에서 불러오는 파이썬 파일을 '모듈'이라 부른다. 다음과 같이 사용한다.

```
>>> import random
```

random.py 파일을 불러 왔다. 그런데 random.py는 어디에 있는 것일까? 이 파일은 기본 내장된 파일로써 파이썬이 설치된 곳에 있다. 그 외에도 파이썬이 지정한 path에 있는 파이썬 파일은 언제든 불러올 수 있다. 다음과 같은 명령으로 path를 알 수 있다.

```
>>> import sys
>>> sys.path
['', '/Users/peter/.pyenv/versions/3.5.3/lib/python35.zip', '/Users/peter/.pyenv/versions/3.5.3/lib/python3.5', '/Users/peter/.pyenv/versions/3.5.3/lib/python3.5/plat-darwin', '/Users/peter/.pyenv/versions/3.5.3/lib/python3.5/lib-dynload', '/Users/peter/.pyenv/versions/gym3/lib/python3.5/site-packages']
```

위 경로 중 한 곳에 자신이 만든 파이썬 파일을 넣어두면 언제든 사용 할 수 있다.

이 path에 자신만의 디렉터리를 추가할 수도 있다.

```
>>> sys.path.append("/Users/peter/mypackage")
```

주의할 점은 윈도우인 경우 "c:/temp" 혹은 "c:\temp"처럼 사용해야 한다. "c:\temp"는 인식되지 않는다.

또한 운영체제마다 경로 지정 방법도 다르다. 그렇기 때문에 다음 함수를 사용하는 것이 좋다.

```
>>> import os
>>> os.getcwd() #현재 워킹 디렉토리를 반환한다.
'C:\\Users\\peter'
>>> os.path.join(os.getcwd(), "temp")
'C:\\Users\\peter\\temp'
```

자신이 만든 파이썬 파일 불러오기
현재 열려 있는 파이썬 파일은 같은 디렉터리에 있는 파이썬 파일을 불러올 수 있다.

```
#my.py
def su(a, b):
    r = a + b
    return r

def su2(a, b):
    r = a - b
    return r

#main.py
>>> import my
>>> print(my.su(3,4))
7
```

import 하는 다양한 방법
my.py를 import 하면 my.su()처럼 항상 my 네임스페이스를 붙여야 한다. 네임스페이스를 바꿔보자.

```
>>> import my as m
>>> m.su(3,4)
7
```

as를 이용해 네임스페이스를 바꿀 수 있다.

사람들이 많이 사용하는 패키지 혹은 모듈을 통상적으로 쓰는 네임스페이스가 있다.

```
>>> import tensorflow as tf
>>> import numpy as np
>>> import pandas as pd
```

되도록 유명한 모듈은 통상적으로 사용되는 네임스페이스를 사용하는 것이 좋다.

때에 따라서는 모듈 안에 있는 특정 함수만 사용하고 싶을 때가 있다. 이럴 땐 from 예약어를 사용한다.

```
>>> from my import su
>>> su(3,4)
7
```

네임스페이스 없이 바로 함수를 사용했다. 네임스페이스가 없다면 다른 모듈의 함수명과 겹치는 경우가 발생 할 수 있으니 주의해야 한다.

그리고 my 모듈 안에 있는 모든 함수를 그대로 불러오고 싶다면 다음과 같이 한다.

```
>>> from my import *
>>> print(su(3,4))
7
>>> print(su2(3,4))
-1
```

more
파이썬 파일은 다른 파이썬 파일에 import되어 모듈로써 사용될 수 있다. 하지만 이런 것을 원하지 않을 수 있다. 어떤 파이썬 파일은 다른 곳에 모듈로 사용되지 않고 최종 파일로써 실행되길 원할 수 있기 때문이다.

이런 경우 다음과 같이 한다.

```
#my.py
def su(a, b):
    if __name__ == "__main__":
        r = a + b
        return r
#결과
from my import *
print(su(1,2))
None
```

my.py파일은 다른 파이썬 파일에 import되어도 su() 함수를 사용할 수 없다. if 문을 통과하지 못하기 때문이다.

파이썬 파일이 만들어지면 name라는 변수가 생긴다. 파이썬 파일을 바로 실행할 때 name 변수의 값은 "main"이 된다. 그러나 다른 곳에 모듈로 사용되면 파일명이 값이 된다.

마지막 편집일시 : 2018년 7월 24일 8:43

30_외부 패키지 설치하기
파이썬에는 이미 잘 만들어진 패키지가 많다. 이런 패키지를 아주 손쉽게 설치, 사용할 수 있다. 외부 패키지를 설치하기 위해서는 파이썬이 아닌 운영체제 쉘에서 명령해야 한다. 데이터 분석 시 많이 사용하는 numpy와 tensorflow 패키지를 설치해 보자.

```
user$> pip install numpy
user$> pip install tensorflow
```

알아서 내려받고 설치까지 완료한다. 파이썬을 실행하고 테스트보자.

```
>>> import numpy
>>> numpy.mean([1, 3, 7])
3.7.06666666666665
```

마지막 편집일시 : 2018년 7월 24일 8:43

31_파일 관리(읽기, 쓰기)
텍스트 파일을 불러오거나 쓸 수 있다. 다음과 같은 텍스트 파일이 있다.

```
#text.txt
Hello World!
```

Good Morning!
Good Afternoon!
Good Evening!
이 파일을 파이썬에서 불러오려면 다음과 같이 한다.

```
f = open("test.txt", 'r')
print(f.readline())
f.close()
#결과
Hello World!
```

readline() 함수를 이용했다. 하지만 이 함수로 읽으면 한 줄밖에 읽어오지 못한다. 하지만 한 번 더 실행하면 다음 줄을 읽어온다.

```
f = open("test.txt", 'r')
print(f.readline())
print(f.readline())
f.close()
#결과
Hello World!
Good Morning!
```

모든 줄을 읽어오려면 반복문을 사용해야 한다.

```
f = open('test.txt', 'r')
while True:
    l = f.readline()
    if not l: break
    print(l)
f.close
#결과
Hello World!
Good Morning!
Good Afternoon!
Good Evening!
```

readline() 함수는 텍스트 파일의 값을 차례대로 불러오기 때문에 한 줄씩 어떤 이벤트에 의해 불러와야 할 때 용이하다.

여러 줄을 한 번에 읽어오는 함수도 있다. **readlines()** 함수이다.

```
f = open("test.txt", 'r')
t = f.readlines()
print(t)
f.close()
#결과
['Hello World!\n', 'Good Morning!\n', 'Good Afternoon!\n', 'Good Evening!']
```

readlines() 함수는 리스트 형식으로 모든 줄의 텍스트를 불러온다. readlines() 함수는 리스트로 값을 받기 때문에 다시 줄 단위로 나눠야 한다면 for 문을 이용하면 된다.

```
f = open("test.txt", 'r')
ll = f.readlines()
for line in ll:
    print(line)
f.close()
#결과
```

Hello World!
Good Morning!
Good Afternoon!
Good Evening!

파일 쓰기(한 줄만 쓰기)

```
f = open("test2.txt", 'w')
f.write("Good Afternoon!")
f.close()
#결과
Good Afternoon!
```

open() 함수 사용 시 두 번째 인자에 'w'가 사용되었다. '쓰기 모드'란 의미이다. 이 인자에는 다음과 같은 값을 사용할 수 있다.

r 읽기모드 w 쓰기모드, 쓰려는 파일이 있다면 모든 내용을 덮어씌운다 x 쓰기모드, 쓰려는 파일이 있으면 오류가 발생한다. a 쓰기모드, 쓰려는 파일이 있으면 뒤에 내용을 추가한다.

파일 쓰기(여러 줄 쓰기)

```
f = open("test2.txt",'w')
f.writelines(["Good Afternoon!\n", "Good Night!\n"])
f.close()
#결과
Good Afternoon!
Good Night!
```

write() 함수와 writelines() 함수가 다르게 사용된 점을 기억하자.

more
파일 내용 그대로 출력하기
open() 함수로 파일을 불러오면 read() 함수를 사용할 수 있다. 이 함수는 텍스트 내용을 그대로 출력해준다.

```
f = open("test.txt", 'r')
t = f.read()
print(t)
f.close()
#결과
Hello World!
Good Morning!
Good Afternoon!
Good Evening!
```

파일에 내용 추가하기

```
f = open("test.txt",'a')
f.write("\nGood Night!")
f.close()
#결과
Hello World!
Good Morning!
Good Afternoon!
Good Evening!
Good Night!
```

마지막 편집일시 : 2018년 7월 24일 8:46

32_Context Managers(with statement)
앞서 open() 함수를 통해 파일을 불러왔다. open() 함수를 사용하면 최종적으로 불러온 파일을 닫아줘야 한다. 그래서 f.close()를 사용했다. 하지만 파일이 불러오면서 에러가 발생할 수도 있다. 지금은 간단한 예이지만 큰 시스템에 적용된 경우라면 에러가 발생되었을 때 사후처리를 제대로 해야 한다. 그러기 위해 반드시 열었던 파일 혹은 시스템을 닫아줘야 한다. 이런 상황이라면 보통 예외 처리를 떠올린다. 올바른 방법이다. 하지만 좀 더 단순하게 구현하고 싶다면 with 문을 사용하면 좋다.

```
with open('test.txt', 'r') as f
    f.readline()
```

with 문을 사용하면 알아서 열고 닫는다. 앞서 만든 코드 보다 훨씬 간결해졌다. 파일은 f라는 변수로 열리고 f.readline()명령을 실행 후 자동으로 닫힌다. with 문이 작동하는 이유는 with문이 실행하는 class에 enter메서드와 exit가 있기 때문이다.

```
f = open('test.txt', 'r')
print(dir(f))
['_CHUNK_SIZE', '__class__', '__del__', '__delattr__', '__ dict__', '__dir__', '__doc__', '__enter__', '__eq__', '__exit__', '__format__', '__ge__', '__getattribute__', '__getstate__', '__
...이하 생략...
```

f의 내용을 보면 enter와 exit가 정의되어 있음을 알 수 있다. with 문을 사용하려면 반드시 객체에 enter메서드와 exit메서드가 구현되어 있어야 한다. 즉, with 문은 이 두 메서드를 차례대로 실행하는 것이다. 머신러닝에 많이 사용하는 tensorflow도 세션을 열고 닫을 때 다음과 같이 사용한다.

```
with tf.Session() as sess:
    이것저것
```

tensorflow의 Session()에도 enter와 exit가 정의되어 있다. 이것을 활용하면 우리도 with문에서 작동하는 class를 만들 수 있다.

```
class test:
    def __enter__(self):
        print("enter")
        a = 'hello'
        return a

    def __exit__(self, exc_type, exc_val, exc_tb):
        print("exit")

with test() as t:
    print(t)

#결과
enter
hello
exit
```

enter메서드와 exit메서드를 정의했다. test()에 의해 먼저 enter() 메서드가 실행된다. 이 안에 있는 "enter"문자열이 가장 먼저 출력된다. 그 후 "hello" 문자열이 반환되어 with의 print()에 전달되어 출력된다. 그리고 with 문의 모든 코드가 실행되어 마지막으로 exit()가 실행되어 "exit"가 출력되었다.
마지막 편집일시 : 2018년 7월 24일 8:47

33_변수를 바이너리 파일로 저장하기(**pickling**)
파이썬은 텍스트 파일 뿐만 아니라 바이너리 형식으로 데이터를 저장하거나 불러올 수 있다. 이런 방식을 파이썬에선 pickling이라 부른다.

데이터를 바이너리로 파일에 저장하기
```
import pickle

data1 = ["elsar", 24, 171]
data2 = 25

with open('test.pickle', 'wb') as f:
    pickle.dump(data1, f)
    pickle.dump(data2, f)
```
바이너리로 저장하기 위해 'wb' 인자를 넣었다. 변수를 저장하기 위해서는 dump() 함수를 사용하는데 변수마다 dump() 함수를 사용하면 된다.

데이터 불러오기
```
import pickle
with open('test.pickle', 'rb') as f:     #'rb'라는 점을 주의하라
data1 = pickle.load(f)
data2 = pickle.load(f)

print(data1)
print(data2)
```
불러올 때는 load() 함수를 사용한다. open() 함수에는 'r'이 아니라 'rb'를 사용해야 한다. 바이너리이기 때문이다. 저장된 변수 형식 그대로 다시 불러올 수 있으므로 작업을 이어서 할 수 있고 다른 작업자에게 쉽게 전달 할 수 있다. 중요한 점이 있다. 알 수 없는 곳에서 받은 피클 파일은 절대 열지 말아야 한다. pickle이 load 되면서 특정 함수가 실행되는데 여기에 악성코드를 심을 수 있다.
마지막 편집일시 : 2018년 7월 24일 8:49

34_변수를 파일로 저장하기(shelve)
변수를 외부 파일에 바이너리로 저장하는 방법은 pickle, shelve 두 가지가 있다. shelve는 저장 시 key값과 매치시켜 저장되기 때문에 shelve.open() 하면 다음부터는 단순히 key값을 변수에 Access할 수 있어 편리하다. 큰 용량의 데이터를 save, access하기 좀 더 쉽다.

데이터 저장하기

```
import shelve
data1 = ["elsar", 24, 171]
data2 = ["peter", 31, 175]

with shelve.open('test.db') as f:
    f['obv1'] = data1
    f['obv2'] = data2
```

데이터 불러오기
```
import shelve
with shelve.open('test.db') as f:
    print(f['obv1'])
    print(f['obv2'])
```
마지막 편집일시 : 2018년 7월 24일 8:59

35_XML 만들기
여러 방법이 있지만, 다음과 같은 방법을 추천한다.

노드 만들기
```
from xml.etree.ElementTree import Element, dump

node1 = Element("first")
node1.text = "안녕"
dump(node1)
#결과
<first>안녕</first>
```

Element() 함수로 노드를 만들고 dump() 함수로 값을 넣으면 끝

노드에 노드 추가하기
```
from xml.etree.ElementTree import Element, dump

root = Element("xml")
node1 = Element("first")
node1.text = "안녕"
root.append(node1)

node2 = Element("second")
node2.text = "Hello"
root.append(node2)
dump(root)

#결과
<xml><first>안녕</first><second>Hello</second></xml>
```
어떤 노드 밑으로 노드를 붙이려면 append() 함수를 이용한다.

속성 추가하기
```
from xml.etree.ElementTree import Element, dump
root = Element("xml", kind="language")
node1 = Element("first")
node1.text = "안녕"
root.append(node1)

node2 = Element("second")
node2.text = "Hello"
root.append(node2)
dump(root)

#결과
<xml kind="language"><first>안녕</first><second>Hello</second></xml>
```
Element() 함수에 인자를 추가하면 해당 노드의 속성으로 추가된다.

보기 좋게 xml만들기
xml은 줄 바꿈과 들여쓰기가 되어 있어야 보기 편하므로 별도 작업이 필요하다.

```
from xml.etree.ElementTree import Element, dump

root = Element("xml", kind="language")
```

```
node1 = Element("first")
node1.text = "안녕"
root.append(node1)

node2 = Element("second")
node2.text = "Hello"
root.append(node2)

def indent(elem, level=0): #자료 출처 https://goo.gl/J8VoDK
    i = "\n" + level*"  "
    if len(elem):
        if not elem.text or not elem.text.strip():
            elem.text = i + "  "
        if not elem.tail or not elem.tail.strip():
            elem.tail = i
        for elem in elem:
            indent(elem, level+1)
        if not elem.tail or not elem.tail.strip():
            elem.tail = i
    else:
        if level and (not elem.tail or not elem.tail.strip()):
            elem.tail = i
indent(root)
dump(root)

#결과
<xml kind="language">
    <first>안녕</first>
    <second>Hello</second>
</xml>
```

indent()함수가 추가되었다. 이 함수만 별도 저장해 모듈로 불러와 사용하면 좋다.

xml파일로 쓰기

```
from xml.etree.ElementTree import ElementTree

ElementTree(note).write("note.xml")
```

more

xml.py을 만들어 사용한다면 에러가 발생할 것이다. 파일 이름과 모듈 이름이 겹치기 때문이다. 두 이름이 겹치지 않도록 해야 한다.

attribute 추가하는 다른 방법

root.attrib["kind"] ="language"

노드에 노드를 추가하는 다른 방법

SubElement(root, "second").text = "hello"

SubElement() 함수를 사용하면 노드에 노드를 추가할 수 있는데 붙여지게 될 노드와 붙일 노드를 각각 인자로 넣으면 된다. 그때 해당 노드의 값도 함께 넣을 수 있다.

마지막 편집일시 : 2018년 7월 24일 9:01

36_XML 파싱

data.xml
```
<?xml version="1.0"?>
<data>
<student>
    <name>peter</name>
    <age>24</age>
    <score math="80" english="97"/>
</student>
<student>
    <name>elgar</name>
    <age>21</age>
    <score math="67" english="56"/>
</student>
<student>
    <name>hong</name>
    <age>36</age>
    <score math="76" english="81"/>
</student>
</data>
```
위 파일을 다음과 같이 파싱 해보자.

xml 파일 파싱
```
from xml.etree.ElementTree import parse

tree = parse('test.xml')
root = tree.getroot()

student = root.findall("student")

name = [x.findtext("name") for x in student]
age = [x.findtext("age") for x in student]
score = [x.find("score").attrib for x in student]

print(name)
print(age)
print(score)
```
```
['peter', 'elgar', 'hong']
['24', '21', '36']
[{'math': '80', 'english': '97'}, {'math': '67', 'english': '56'},
{'math': '76', 'english': '81'}]
```
getroot() 함수로 xml를 불러온 후 findall() 함수로 원하는 노드를 불러온다.

xml 파일 가져오기
tree = parse('파일명.xml')

root 노드 가져오기
root = tree.getroot()

일치하는 모든 노드 가져오기
root.findall("student")
"root" 하위 노드 중 "student" 이름을 가진 모든 노드를 가져온다.

일치하는 첫 번째 노드 가져오기
student.find("name")
<name>peter</name>
"student" 하위 노드 중 "name"이름을 가진 첫 번째 노드를 가져온다. 값이 아닌 노드를 가져온다는 점을 기억해야 한다. 만약 값을 가져오고 싶다면

student.find("name").text
이렇게 해야 한다.

일치하는 첫 번째 노드의 값 가져오기
student.findtext("name")

#결과
<name>peter</name>
"student" 하위 노드 중 "name"이름을 가진 첫 번째 노드의 값을 가져온다. 즉, 아래 두 코드는 결괏값이 같다.

student.find("name").text
student.findtext("name")
일치하는 첫 번째 노드의 속성 가져오기
student.findtext("score").attrib

#결과
<score math="76" english="81"/>
"student"하위 노드 중 "score"이름을 가진 첫 번째 노드의 속성값 가져온다.

마지막 편집일시 : 2018년 7월 24일 9:04

38_json 파싱
다음과 같이 json 파일을 파싱할 수 있다. 크게 복잡한 것은 없다. json 모듈의 load() 함수만 사용하면 된다.

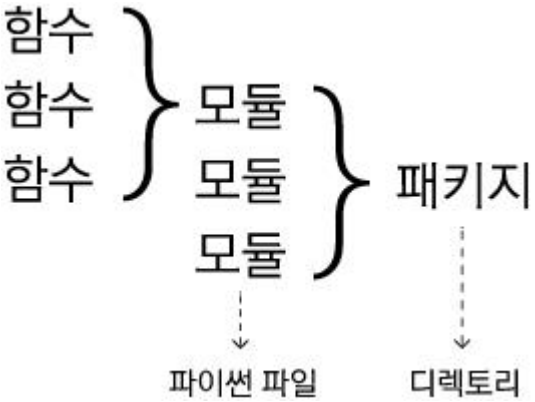
```
import json #json을 다루기 위한 모듈
from collections import OrderedDict
from pprint import pprint #json을 좀 더 보기 편하게 출력

with open('test.json', 'r') as f:
    data = json.load(f, object_pairs_hook=OrderedDict)
pprint(data)

#결과
OrderedDict([('student1',
    OrderedDict([('age', 24),
        ('name', 'peter'),
        ('score',

OrderedDict([('math', 80), ('english', 97)]))])),
        ('student2',
            OrderedDict([('age', 21), ('name', 'elgar'), ('score', 80)])),
        ('student3',
            OrderedDict([('age', 36), ('name', 'hong'), ('score', 80)]))])
```

마지막 편집일시 : 2018년 7월 24일 9:06

==========

03_더 깊게

01_패키지 만들기



패키지는 단순히 모듈의 모음이다. 패키지를 자신이 편하게 사용하기 위해 만들 수도 있고 배포하기 위해서 만들 수도 있는데 만약 배포를 목적으로 한다면 모듈들을 디렉터리별로 나누고 버젼 관리를 하고 다른 패키지의 의존성, 도움말 등을 정의해야 한다. 할 것들이 꽤 많다. 여기서는 단순히 자신이 편리하게 사용하기 위한 목적으로 패키지를 만드는 방법을 살펴보자. 즉, 패키지를 만든다는 것은 단순히 모듈들을 디렉터리로 정리하는 것이다. 모듈 이름이 파일 이듯이 패키지 이름도 디렉터리 이름일 뿐이다. 먼저 working directory를 확인한다.

```
>>> import os
>>> os.getcwd()
```

working directory 밑에 hello라는 디렉터리를 만든다. hello 디렉터리에 morning.py파일을 만들고 다음 내용을 넣는다.

```
def test():
    print("Good Morning!")
```

working directory에 main.py를 만들고 실행해 보자.

```
import hello.morning as mn
mn.test()

#결과
Good Morning!
```

중요한 점은 working directory 기준으로 하위 디렉터리로 패키지가 인식된다는 점이다. 만약 자신이 만든 모듈을 설치된 패키지처럼 자유롭게 사용하고 싶다면 아래 목록에 있는 디렉터리에 넣으면 된다.

```
>>> import sys
>>> sys.path
['', '/Users/peter/.pyenv/versions/3.5.3/lib/python35.zip',
'/Users/peter/.pyenv/versions/3.5.3/lib/python3.5', '/Users/
peter/.pyenv/versions/3.5.3/lib/python3.5/plat-darwin', '/
Users/peter/.pyenv/versions/3.5.3/lib/python3.5/lib-dynload',
'/Users/peter/.pyenv/versions/gym3/lib/python3.5/site-
packages']
```

혹은 /User/peter/mypackage 를 만들고 이 안에 넣어 관리하고 싶다면 이 디렉터리 경로를 추가하면 된다.

```
>>> import sys
>>> sys.path.append("/Users/peter/mypackage
```

이 안에 hello 디렉터리를 옮기고 테스트하면 제대로 인식되는 것을 확인할 수 있다.

마지막 편집일시 : 2018년 7월 22일 9:09

02_예외 처리(Exception)
다음과 같은 명령을 하면 에러가 발생한다.

```
>>> 4/0
ZeroDivisionError: division by zero
```

0으로 나눌 수 없기 때문이다. 하지만 다음과 같이 하면 에러가 발생하지 않고 그냥 넘어간다

```
try:
    4/0
except:
    pass
```

try문은 에러가 있어도 무조건 실행된다. 실행 중 에러가 발생하면 except문이 실행되는데 위 코드의 경우 큰 권력자인 pass가 에러를 그냥 넘어가란다.

이런 경우를 예외 처리라고 부르는데 에러에 눈감아주기 위해 사용되는 경우도 있지만, 이 에러에 대해 특별히 처리하기 위해서 사용한다. 좋은 프로그램은 오류가 발생했을 때 어떤 조치를 취한다.

```
try:
    print("무조건 실행")
    4 / 0   #Python에선 0으로 나눌 수 없음.
    print("end")
except :
    print("Error 발생")

#결과
무조건 실행
Error 발생
```

에러가 발생하면 즉시 except문을 실행한다. 따라서 위 코드 중 try문의 세 번째 명령문이 실행되지 않았다.

정해진 오류에만 반응하기

```
>>> 3/0
Traceback (most recent call last):
 File "<stdin>", line 1, in <module>
ZeroDivisionError: division by zero
```

0으로 나누게 되면 발생하는 오류의 이름은 'ZeroDivisionError'이고 메시지는 'division by zero'이다. 이런 오류에만 반응하도록 예외 처리하는 방법은 다음과 같다.

```
try:
    3 / 0
except ZeroDivisionError as e:
    print(e)
```

#결과
division by zero

else문 사용하기
```
try:
    4 / 1
except :
    print("Error")
else :
    print("OK")
```

#결과
OK
위와 같이 하면 예외가 발생한 경우와 발생하지 않은 경우를 각각 처리할 수 있다.

finally문 사용하기
```
try:
    4 / 0
except :
    print("Error")
else :
    print("OK")
finally :
    print("수고했어")
```
#결과
Error
수고했어
finally문은 에러가 발생하든 발생하지 않든 무조건 실행된다.

more
```
try:
    a = ["apple", "orange", "banana"]
    print(a[4])
    3/0
except (ZeroDivisionError, IndexError) as e:
    print(e)
```
위 코드처럼 두 개의 오류에 대해 처리할 수도 있다.

마지막 편집일시 : 2018년 7월 22일 10:30

03_가정 설정문(assert)
assert는 뒤의 조건이 True가 아니면 AssertError를 발생한다.

```
>>> a = 3
>>> assert a == 2
```

#결과
```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AssertionError
```
왜 assert가 필요한 것일까?

어떤 함수는 성능을 높이기 위해 반드시 정수만을 입력받아 처리하도록 만들 수 있다. 이런

함수를 만들기 위해서는 반드시 함수에 정수만 들어오는지 확인할 필요가 있다. 이를 위해 if 문을 사용할 수도 있고 '예외 처리'를 사용할 수도 있지만 '가정 설정문'을 사용하는 방법도 있다.

아래 코드는 함수 인자가 정수인지 확인하는 코드이다.

```
lists = [1, 3, 6, 3, 8, 7, 13, 23, 13, 2, 3.14, 2, 3, 7]

def test(t):
    assert type(t) is int, '정수 아닌 값이 있네'

for i in lists:
    test(i)
#결과
AssertionError: 정수 아닌 값이 있네
```

lists에 실수가 하나 있으므로 AssertionError가 발생했다. assert 문은 다음 형식으로 작동한다.

```
assert 조건, '메시지'
```

'메시지'는 생략할 수 있다.

assert는 개발자가 프로그램을 만드는 과정에 관여한다. 원하는 조건의 변수 값을 보증 받을 때까지 assert로 테스트 할 수 있다.

이는 단순히 에러를 찾는 것이 아니라 값을 보증하기 위해 사용된다.

예를 들어 함수의 입력 값이 어떤 조건의 참임을 보증하기 위해 사용할 수 있고 함수의 반환 값이 어떤 조건에 만족하도록 만들 수 있다. 혹은 변수 값이 변하는 과정에서 특정 부분은 반드시 어떤 영역에 속하는 것을 보증하기 위해 가정 설정문을 통해 확인 할 수도 있다.

이처럼 실수를 가정해 값을 보증하는 방식으로 코딩하기 때문에 이를 '방어적 프로그래밍'이라 부른다.

마지막 편집일시 : 2018년 7월 22일 10:33

04_클래스(class)
class는 객체지향성 언어의 개념이다. 틀을 만들어 찍어내는 공장과 거의 비슷하다. 틀이 class이고 틀로 찍어낸 제품이 '객체'이다. 파이썬에서도 class와 객체를 만들 수 있다. 다만 Java처럼 문법이 엄격하진 않다. 먼저 class를 만들자. class라는 예약어를 사용하면 된다.

```
class temp:
    a = None

    def __init__(self, str):  # 객체가 만들어질 때 반드시 실행되는 메서드
        self.a = str

    def driving(self, str):  # 메서드이며 반드시 첫 번째 인자에 self를 넣어야 한다
        print("%s로 운전 중입니다" % (str))

tory = temp("solid")
print(tory.a)
tory.driving("서울")
```

#결과
solid
서울로 운전 중입니다

몇 가지 규칙이 있다. init()메서드는 객체가 만들어지면서 실행되는 메서드이다. 이 안에 명령어를 넣으면 인스턴스가 만들어질 때 반드시 실행된다. 메서드를 만들 때 첫 번째 인자로 반드시 self를 사용해야 한다. 여기서 self가 가리키는 것은 'class'가 아니라 '객체'이다. 메서드에는 따로 private나 public이 있지 않다. 은닉 문제가 생길 수 있는데 이 문제는 setter/getter를 다룰 때 자세히 다룬다.

more
인스턴스가 생성된다는 것은 class에 bound 된다는 의미이다. 무슨 의미인지 예제를 통해 살펴보자. 다음 코드는 자동차를 만드는 class이다. 그리고 자동차의 힘을 나타내는 값 중 하나인 마력인 horpower 변수가 있다.

```
class Car:
    def __init__(self, num):
        self.horpower = num

    def get_horsepower(self):
        return self.horpower
```

class의 메서드는 class 속성 중 하나이다. 다음과 같이 명령하면 그것을 알려준다.

```
>>> print(Car.get_horsepower)
<function Car.get_horsepower at 0x10d802b70>
```

하지만 위 메서드는 아직 unbound된 메서드라서 아무것도 할 수 없다. 한 마디로 빈 껍데기이다.

```
>>> print(Car.get_horsepower())
TypeError: get_horsepower() missing 1 required positional argument: 'self'
```

에러가 나는 이유는 get_horsepower에서 요구하는 인자인 인스턴스가 전달되지 않았기 때문이다. 그래서 만약 다음과 같이 인스턴스를 넣어주면 제대로 작동한다.

```
>>> print(Car.get_horsepower(Car(120)))
120
```

근데 class 메서드에 직접 인스턴스를 전달하지 않아도 작동한다.

```
>>> print(Car(120).get_horsepower())
120
```

인자가 전달되지 않았는데도 제대로 작동한다. 그 이유는 객체가 만들어지면 해당 메서드가 자동으로 연결(bound)된다는 것을 의미한다. 다음과 같이 인스턴스에 의해 만들어진 메서드에 대해 출력해보면 알 수 있다.

```
>>> print(Car(120).get_horsepower)
<bound method Car.get_horsepower of <__main__.Car
object at 0x10a21b2e8>>
```

bound 된 메서드라고 알려준다. 그래서 문자열을 대문자로 바꿔주는 upper()를 자주 보던 다음과 같은 두 가지 방식으로 명령할 수 있다.

```
>>> s = 'hello'
>>> s.upper()
'HELLO'
>>> str.upper(s)
```

'HELLO'
그리고 한번 bound 되면 get_horsepower에 인자를 전달하지 않아도 된다. 자동으로 전달된다.

```
>>> a = Car(120).get_horsepower
>>> print(a())
120
>>> b = str(s).upper
print(b())
HELLO
```

마지막 편집일시 : 2018년 7월 22일 10:36

05_상속(Inheritance)
상속은 기존 class를 그대로 물려받아 업그레이드시켜 사용하는 방법이다. 상속받는 방법은 쉽다. 여기 상속해줄 class가 있다.

```
class Car:
    kind = "Sedan"
    countOfDoors = 5

    def __init__(self, str):
        self.name = str

    def sethorsepower(self, str): #setter
        self.horsepower = str

    def driving(self, str):
        print("%s로 운전 중입니다" % (str))
```

상속 받아보자.

```
class Car_nextGeneration(Car):  #Car클래스를 상속
    autoPilot = True     #새롭게 추가된 속성
    countOfAirbag = 10; #새롭게 추가된 속성

    def setwheelSize(self, str): #새롭게 추가된 setter
        self.wheelSize = str

    def driving(self, str): #메서드 오버라이딩
        print("%s(으)로 자동 운전 중입니다" % (str))

    def turnning(self, num): #새롭게 추가된 메서드
        self.turnninghorsepower = self.horsepower + num
        return self.turnninghorsepower
```

만드는 class 이름에 상속하고 싶은 class를 명시하기만 하면 된다. 모두 구현되었다. 이제 객체를 만들어보자.

```
toch = Car("touch")
toch.horsepower = 120
toch.driving("서울")
#결과
서울로 운전 중입니다
```

이제 새롭게 추가된 메서드를 사용할 수 있다.

```
Mbbang = Car_nextGeneration("Mbbang")
Mbbang.horsepower = 300
Mbbang.wheelSize = 20
Mbbang.turnning(200)
Mbbang.driving("인천")
#결과
인천(으)로 자동 운전 중입니다
```
마지막 편집일시 : 2018년 7월 22일 10:38

06_Setter 와 Getter
파이썬에서 class을 지원하기 때문에 setter/getter 또한 지원한다. 기본적으로 다음과 같이 구현할 수 있다. 메서드에 set 혹은 get을 붙이면 된다.

```
class Car:
    def __init__(self, t):
        self.horsepower = t

    def gethorsepower(self):          # getter
        return self.horsepower

    def sethorsepower(self, str):    # setter
        self.horsepower = str
```
하지만 이렇게 하면 car.sethorsepower()에 직접 값을 넣을 수도 있고 car.gethorsepower()를 직접 사용할 수도 있다. 즉, 은닉되지 않는다. 파이썬에는 private 혹은 public과 같은 예약어가 없기 때문이다. 따라서 메서드를 은닉하기 위해 다음과 같이 한다.

```
class Car:
    def __init__(self):
        self.__horsepower = 100

    @property
    def horsepower(self):    #getter
        return self.__horsepower

    @horsepower.setter
    def horsepower(self, str): #setter
        self.__horsepower = str
```
먼저 init()메서드에 변수를 정의하면서 __ 기호를 붙여준다. 그 후 decorator를 붙여준다. 이렇게 하면 은닉할 수 있다.

마지막 편집일시 : 2018년 7월 22일 10:38

07_정적 메소드(Static Method, Class Method)
정적메서드를 지원하는 두 가지 방법이 있다.

@staticmethod @classmethod

같은 면과 다른 면을 각각 살펴보자.

먼저 같은 점은 둘 다 인스턴스를 만들지 않아도 class의 메서드를 바로 실행할 수 있다는 것이다.

```
#staticmethod
class hello:
    num = 10

    @staticmethod
    def calc(x):
        return x + 10

print(hello.calc(10))
#결과
20
#classmethod
class hello:
    num = 10

    @classmethod
    def calc(cls, x):
        return x + 10

print(hello.calc(10))
#결과
20
```

둘 다 객체를 만들지 않고 바로 해당 메서드를 사용했다. 차이점은 calc메서드를 만들 때 cls라는 인자가 더 추가 된 점이다. 잠시 뒤 cls의 정체를 알게 될 것이다.

이제 개념적인 차이점을 살펴보자. 만약 hello 클래스의 num 속성에 접근하려면 어떻게 해야 할까? 우선 객체로 접근하는 것이 아니기 때문에 self.num를 사용할 순 없다. 억지로 사용하고 싶다면 @staticmethod는 다음과 같이 해야 한다.

```
#staticmethod
class hello:
    num = 10

    @staticmethod
    def calc(x):
        return x + 10 + hello.num

print(hello.calc(10))
#결과
30
```

정적 변수로 접근했다. 좋아 보이지 않는다. @classmethod는 좀 더 세련된 방법으로 접근한다.

```
#classmethod
class hello:
    num = 10

    @classmethod
    def calc(cls, x):
        return x + 10 + cls.num

print(hello.calc(10))
#결과
30
```

classmethod는 cls가 있는데 이것은 '클래스'를 가리킨다. 이것으로 클래스의 어떤 속성에도 접근할 수 있다. 위 예시 경우 또한 cls.num을 통해 hello 클래스의 num 속성에 접근했다.

만약 상속 관계가 있는 클래스들에선 cls가 가리키는 클래스는 어떤 클래스일까?

```
class hello:
    t = '내가 상속해 줬어'

    @classmethod
    def calc(cls):
        return cls.t

class hello_2(hello):
    t = '나는 상속 받았어'

print(hello_2.calc())
#결과
내가 상속해 줬어
```

상속받은 hello_2 클래스가 t 속성을 업데이트 했다. cls.t이 상속시켜준 클래스에 있더라도 이것이 가리키는 것은 상속받은 클래스의 t 속성이다. cls는 상속 받은 클래스에서 먼저 찾는다.

그렇다고 @staticmethod는 필요 없는 것은 아니다. 사람마다 다르겠지만 유틸리티 성격에 맞게 사용할 수 있을 것이다.

그리고 @classmethod는 클래스의 어떤 옵션을 바꾸는 용도로 사용할 수 있다.

마지막 편집일시 : 2018년 7월 22일 10:46

08_추상 메서드(Abstract class)
잘 만들어진 class는 잘 사용되도록 만들어져있다. class로 만들었다면 class에서 정의된 메서드를 있는 그대로 사용하지 않을 것이다. 반드시 상속받아 class변수를 추가하거나 메서드를 추가할 것이고 어떤 메서드는 오버라이드 해야 할 것이다. 그래야만 class로 만든 의미가 있다. 확장된 기능을 만들면서 통일된 체계를 구축하는 것이 class의 개념이다.

이런 개념을 유지하기 위해 class를 만들 때 반드시 구현 해야 할 메서드를 class에 명시할 수 있다.

```
class Car:
    def turnning(cls, horsepower):
        raise NotImplementedError

class Sonaty(Car):
    def turnning(self):
        print("turnning finish")

sonaty = Sonaty()
sonaty.turnning()
```

위와 같이 하면 상속받은 Sonaty class에서 turnning메서드를 구현하지 않으면 에러가 발생한다. raise NotImplementedError가 이런 역할을 한다.

이보다 좀 더 세련된 방법이 있다.

```
import abc

class Car:
    @abc.abstractmethod
    def turnning(cls, horsepower):
        pass

class Sonaty(Car):
    def turnning(self):
    print("turnning finish")

sonaty = Sonaty()
sonaty.turnning()
```

abs 모듈을 사용했고 @abc.abstractmethod라는 decorator를 사용했다. 이렇게 하면 상속받은 class는 반드시 해당 메서드를 구현해야만 한다. 두 가지 방법 모두 추상화된 것이다. 하지만 두 방법은 차이점이 있다.

첫 번째 방법은 메서드가 호출될 때 비로소 에러가 발생하지만 두 번째 방법은 class가 호출되는 순간부터 에러를 발생시킨다.

그리고 두 번째 방법은 상속시켜준 class를 인스턴스화 시킬 수 없다. 여러 가지 측면에서 두 번째 방법이 좀 더 추상화하는데 이점이 많다.

마지막 편집일시 : 2018년 7월 22일 10:49

09_메타 클래스(MetaClass)
파이썬에선 클래스도 객체이다.

클래스도 객체라면 클래스를 만드는 또 다른 클래스가 있단 말인가?

있다!!

이를 "메타 클래스"라 부른다.

클래스로 객체를 만들 듯 메타 클래스로 클래스를 만들 수 있다는 의미이다.

갑자기 복잡한 개념이 나와서 혼란스러울 수 있다. 걱정할 필요는 없다. 메타 클래스를 쓸 일은 아주 드물다. Django와 같은 프레임워크를 만들어야 한다면 필요하지만 왠만한 프로젝트에서 필수로 사용할 만큼은 아니다.

그래도 알아두면 좋고 파이썬의 구조에 대해 더 자세히 이해할 수 있다.

놀랍게도 우리는 메타 클래스를 이미 알고 있다. 이 녀석은 바로 type()이다.

변수가 어떤 형인지 알려주는 함수라 알고 있던 그 함수이다.

```
>>> type(3)
<class 'int'>
>>> temp = type('temp', (), {})
>>> temp
<class '__main__.temp'>
```

자료형을 알려주던 type()을 이용해 temp라는 클래스를 만들었다. 이런 능력이 있을 줄이야!

int는 정수 객체를 만드는 클래스이다. 근데 메타 클래스로 만들어졌다고 하니 한번 조사해보자.

```
>>> type(int)
<class 'type'>
```
역시 type이라는 Class에 의해 만들어졌다고 알려준다.

그럼 메타 클래스는 어떻게 작동하며 어느 용도로 사용할 수 있는지 살펴보자.

먼저 type() 메타 클래스로 클래스를 만들어보자.

```
>>> temp = type('temp', (), {})
>>> temp
<class '__main__.temp'>
```
(), {}는 각각 튜플과 딕셔너리로 상속 클래스 정의와 클래스에서 정의할 속성과 메서드가 들어간다. 예시를 살펴보자.

다음과 같은 클래스가 있다.

```
class temp:
    a = 3

    def m(self, m, n):
        return m + n

ins = temp()
print(ins.m(3,4))
#결과
7
```
이 클래스를 type()이용해 만들면 다음과 같다.

```
>>> ins = type('temp', (object,), {'a':3, 'm':lambda a, b: a+b})
>>> print(ins.m(3,4))
7
```
두 번째 인자는 상속 시켜줄 부모 클래스이다. (object,)라고 쓰여진 이유는 튜플에 한 개 값만 들어갔기 때문이다. 위 코드의 경우 굳이 object를 상속 받을 필요는 없다.

세 번째 인자 {}부분에 클래스에서 명시할 속성과 메서드가 들어간다. 특히 메서드는 위처럼 labbda를 사용해도 되고 함수를 하나 만들어 변수에 넣고 그 변수에 여기에 명시해도 된다.

가장 중요한 질문, "그래서 메타 클래스로 뭘 할 수 있는데?"

이것에 대한 첫 번째 답변을 얻었다. 위의 예시처럼 우리는 동적으로 클래스를 만들 수 있다. 클래스에 명시되는 속성과 메서드를 딕셔너리에 넣으면 되니 동적으로 만드는 것이 어렵지 않다.

다음으로 메타 클래스로 할 수 있는 것은 커스텀 메타 클래스를 만드는 것이다.

커스텀 메타 클래스를 활용하면 클래스를 컨트롤해 원하는 방향으로 클래스가 생성되게 만들 수 있다.

클래스가 만들어 질 때 특정 속성은 반드시 정수가 되도록 만들어 보자. 검증 과정은 메타 클래스에서 중요한 부분이 될 수 있다.

```
class myMetaclass(type):

    def __new__(cls, clsname, bases, dct):
        assert type(dct['a']) is int, 'a속성이 정수가 아니에요.'
        return type.__new__(cls, clsname, bases, dct)

class temp(metaclass=myMetaclass):
    a = 3.14

ins = temp()
#결과
AssertionError: a속성이 정수가 아니에요..
```

커스텀 메타 클래스를 만들기 위해서는 type 클래스를 상속 받아야 한다. 그리고 type 클래스가 가지고 있는 new메서드를 오버라이드하면 된다. 이 메서드에 하고 싶은 것을 명시하면 된다.

new메서드에 필요한 인자는 4개 인데 마지막 dct인자만 살펴보자. 메타 클래스로 클래스를 만들 때 속성과 메서드를 이곳에 명시하면 된다. type()으로 클래스를 만들 때 맨 끝에 있던 인자와 같다.

assert로 검증을 시도한다. 특히 속성 중 a라는 속성에 대해 정수인지 검증한다. assert에 대해 설명하면서 작성했던 정수를 검증하는 방법을 여기서도 사용했다.

new메서드에서 반환해야 할 값은 type.new() 메서드로 받는 클래스이다.

테스트 하기 위해 메타 클래스를 사용해야 하는데 클래스를 만들면서 metaclass= 라는 인자에 명시하면 된다.

a속성에 실수를 넣어 테스트 해보면 assert가 작동하는 것을 확인할 수 있다.

마지막 편집일시 : 2018년 7월 22일 10:52

10_메모리 관련
메모리 주소 알아내기

```
>>> id(2)
4484212032
```

16진법으로 나타내면 다음과 같다.

```
>>> hex(id(2))
'0x10b47a540'
```

가비지 컬렉션
파이썬은 불필요한 객체를 자동으로 삭제한다. 별도 관리할 필요는 없다. 다만 GC 관련된 기능을 꼭 써야겠다면 모듈을 불러와 사용할 수 있다. 하지만 추천하지 않는다.

```
import gc
```

성능 향상과 메모리를 잘 쓰는 방법
대부분의 프로그래밍 언어가 느려지는 경우는 메모리가 재할당이 이뤄지기 때문이다. 그리고 주로 for 문에서 이런 일이 생긴다. 다음과 같은 경우가 잘못 코딩된 예이다.

```
import time

start_time = time.time()
a = list(range(100000))
```

```
a2 = list()

for i in a:
    a2.append(i*2)

end_time = time.time()
fin = end_time - start_time
print(fin)
#결과
0.04319477081298828
```

위 코드는 for 문의 스텝이 진행될 때마다 메모리 재할당이 일어난다. a2 변수에 append하기 때문이다. 속도저하가 일어나고 메모리를 효율적으로 사용하지 못한다. 결과는 제대로 나와도 이런 코드는 효율적이지 못하다.

더 나은 코드는 아래와 같다. comprehension 방식이다.

```
import time

start_time = time.time()
temp = [x*2 for x in range(100000)]
end_time = time.time()
fin = end_time - start_time
print(fin)
#결과
0.01129007339477539
```

속도가 많이 개선되었다. temp 변수를 새롭게 만들면서 규칙을 명시했다.

하지만 더 나은 코드는 아래와 같다.

```
import time

start_time = time.time()
a = list(range(100000))
a2 = map(lambda n: n*2, a)
end_time = time.time()

fin = end_time - start_time
print(fin)
#결과
0.0034818649291992188
```

속도가 훨씬 빨라졌다. map() 함수를 사용하면 파이썬 내부적으로 연산과 메모리를 관리하기 때문에 효율적으로 모든 요소에 함수를 적용할 수 있다. lambda 예약어를 사용해 함수를 만들고 이 함수를 a 변수에 적용했다. 이 자리에 어떤 함수든 사용할 수 있다.

마지막 편집일시 : 2018년 7월 22일 10:55

==========

04_더 넓게

01_파일, 디렉터리 관리하기
파이썬의 장점 중 하나는 일률적인 업무를 돕는 프로그램을 쉽게 만들 수 있다는 점이다. 이런 것이 가능한 이유는 파일과 디렉터리를 쉽게 관리할 수 있기 때문이다.

파일, 디렉터리 관리와 관련된 패키지는 os, shutil이다. 각각 용도에 따라 다르게 사용된다.

Working Directory 알아내기, 바꾸기
인터프리터 언어 대부분은 working directory가 있다. 현재 작업 중인 디렉터리를 설정하는 것이다. 아무것도 설정하지 않았다면 home일 가능성이 크다. 알고 싶다면 다음과 같이 한다.

```
>>> import os
>>> os.getcwd()
/Users/peter
```

working directory를 바꾸고 싶다면
```
>>> os.chdir("Users/peter/hello")
```

어떤 디렉터리 안의 파일 및 디렉터리 목록 가져오기
```
>>> import os
>>> os.listdir("/Users/peter")
['.android', '.babel.json', '.bash_history', '.bash_profile',
…생략합니다…]
```

디렉터리와 파일 목록을 보여준다. 하지만 하위 디렉터리까지 검사는 하지 않는다.
만약 특정 확장자 파일만 알고 싶다면

```
>>> import glob
>>> glob.glob('./*.py')
```
하지만 이 방법도 하위 디렉터리까지는 검사하지 않는다.

하위 디렉터리까지 검사 하려면 os.walk() 함수를 사용해야 한다.
```
for root, dirs, files in os.walk(os.getcwd()):
    print(files)
```

만약 특정 확장자 파일만 찾고 싶다면 다음과 같이 해야 한다.
```
for root, dir, files in os.walk(os.getcwd()):
    for f in files:
        a = os.path.splitext(f)[-1]
        if a == '.xml':
            print(f)
```

files는 디렉터리별로 파일목록을 가진 변수이다. 이 목록에서 특정 확장자 파일을 찾으면 된다. os.path.splitext() 함수는 파일명과 확장자명을 분리해주는 함수이다. **이 함수는 OS마다 디렉터리 경로 및 파일명 체계가 다르더라도 모두 대비하도록 설계되어 있다.**

파일, 디렉터리 존재 여부 조사하기
파일 혹은 디렉터리가 있는지 검사

```
>>> os.path.exists("/Users/peter")
True
>>> os.path.exists("/Users/test.xml")
```

False

디렉터리만 검사
>>> os.path.isdir("/Users/peter")
True

파일만 검사
>>> os.path.isfile("/Users/peter/test.dmg")
False

파일, 디렉터리 복사, 이동, 삭제하기

복사하기(파일)
>>> import shutil
>>> shutil.copy("data.xml", "data2.xml")
Working Directory안에 있는 data.xml파일을 data2.xml로 복사한다. 디렉터리와 함께 파일명을 명시할 수도 있다. 만약 두 번째 인자에 디렉터리명이 명시되면 디렉터리에 같은 이름으로 복사된다.

복사하기(디렉터리)
>>> import shutil
>>> shutil.copytree("asset", "data")
첫 번째 인자의 디렉터리를 두 번째 인자의 이름대로 새로운 디렉터리를 만들어 복사한다.

이동하기(파일, 디렉터리)
>>> import shutil
>>> shutil.move("data.xml", "data2.xml")
shutil.copy() 함수와 사용방법은 같다. 복사가 아니라 이동인 점만 다르다. 그리고 이 함수는 디렉터리에도 적용할 수 있다. 만약 두 번째 인자의 디렉터리가 없다면 생성되면서 이동된다. 만약 디렉터리가 있다면 그 디렉터리 안으로 이동된다.

삭제하기(디렉터리)
>>> import shutil
>>> shutil.rmtree('asset')

삭제하기(파일)
>>> import os
>>> os.remove('data.xml')

more
파일 크기를 알아내려면
>>> os.path.getsize("/Users/peter/hello.py")

파일 생성, 삭제 감시하기
특정 폴더에 파일이 생성되거나 삭제되는 등 이벤트가 발생하면 이를 감지하고 그때마다 명령어를 실행할 수 있도록 할 수 있다. 이때 사용하는 패키지는 watchdog이다. 아래 코드는 감시부터 이벤트 처리까지의 기본 템플릿이다.

```
import os
import time
from watchdog.observers import Observer
from watchdog.events import FileSystemEventHandler

class Target:
```

```
    watchDir = os.getcwd()
    #watchDir에 감시하려는 디렉토리를 명시한다.

    def __init__(self):
        self.observer = Observer()    #observer객체를 만듦

    def run(self):
        event_handler = Handler()
        self.observer.schedule(event_handler, self.watchDir,
                                                        recursive=True)
        self.observer.start()
        try:
            while True:
                time.sleep(1)
        except:
            self.observer.stop()
            print("Error")
            self.observer.join()

class Handler(FileSystemEventHandler):
#FileSystemEventHandler 클래스를 상속받음.
#아래 핸들러들을 오버라이드 함

    #파일, 디렉터리가 move 되거나 rename 되면 실행
    def on_moved(self, event):
        print(event)

    def on_created(self, event): #파일, 디렉터리가 생성되면 실행
        print(event)

    def on_deleted(self, event): #파일, 디렉터리가 삭제되면 실행
        print(event)

    def on_modified(self, event): #파일, 디렉터리가 수정되면 실행
        print(event)

if __name__ == '__main__': #본 파일에서 실행될 때만 실행되도록 함
    w = Target()
    w.run()
```

on_moved(), on_created() 함수에 원하는 코드를 입력하면 된다.

마지막 편집일시 : 2018년 7월 22일 10:26

02_비동기 프로그래밍(Async)

웹페이지를 불러오는 함수가 있다. 1000페이지를 불러오기 위해 시작하면 한 페이지 불러오고... 또, 한 페이지 불러오고... 또, 한페이지... 불러올 것이다.

이보다 효율적으로 페이지를 불러오기 위해서는

첫째, 1000페이지를 동시에 불러온다.

둘째, 한 페이지를 로딩하면서 기다리는 동안 다른 페이지를 불러온다.

두 가지 방식이 비슷해 보여도 일 처리하는 방식은 다르다. 1번은 멀티스레드 방식이고 2번은 비동기 프로그래밍 방식이다.

이 두 개념의 차이를 이해해보자.

한 회사원이 있다. 전화도 받아야 하고 결재 올려 승인을 받아야 하고 메일도 보내야 하고 고객과 만나 업무도 처리해야 한다.

가끔 바쁠 때 "몸이 열 개면 좋겠다"라고 말한다. 멀티 스레드는 실제 몸이 열 개이다. 그래서 전화 받는 공무원, 메일 보내는 공무원, 민원 처리하는 공무원이 각각 존재한다. 당연히 빠르다. 하지만 전화를 끊은 공무원은 시간이 남아도 다른 일을 도와주지 않는다. 그냥 논다.

비동기식 프로그래밍은 "몸이 열 개면 좋겠다"라고 생각만 하고 결국 혼자 모든 일을 다 한다. 대신 엄청나게 똑똑하고 몸이 빠르다.

상사에게 결재 올려 승인이 날 때까지 기다리지 않고 전화도 받고 메일도 보낸다. 이 분은 '일머리'를 안다. 똑똑하고 체계적이고 계획적이다. 열 명의 공무원 부럽지 않다.

이처럼 비동기식 프로그래밍을 하기 위해서는 한번 주면 받을 때까지 기다리기만 하는 함수를 발전시켜 멀티태스킹이 가능한 함수로 만들어야 한다. 이런 멀티태스킹이 가능한 함수를 특별히 코루틴(Coroutine)이라 부른다.

파이썬 3.5이상에서는 Coroutine을 좀 더 계획적이고 더 나은 환경을 제공한다. 파이썬에 내장된 asyncio 패키지를 사용하는 것이다.

비동기식으로 블로그 포스트를 크롤링 하는 예제를 살펴보자. 아래 코드를 실행하면 여러 페이지를 로딩될 때까지 기다리지 않고 바로 다음 페이지를 로딩하여 아주 빠르게 모든 페이지를 불러온다.

```
import requests
import asyncio
from bs4 import BeautifulSoup
import time
s = time.time()
results = []

#---------------------- A 파트 ------------------------
async def getpage(url):
    req = await loop.run_in_executor(None, requests.get,
                                                                    url)
    html = req.text
    soup = await loop.run_in_executor(None, BeautifulSoup,
                                html, 'lxml')
    return soup

#---------------------- B 파트 ------------------------
async def main():
    urls = ["https://wp.me/p9x2W1-x",
            "https://wp.me/p9x2W1-w",
            "https://wp.me/p9x2W1-t",
            "https://wp.me/p9x2W1-q",
            "https://wp.me/p9x2W1-p",
            "https://wp.me/p9x2W1-j",
            "https://wp.me/p9x2W1-h"]
```

```
    fts = [asyncio.ensure_future(getpage(u)) for u in urls]
    r = await asyncio.gather(*fts)
    global results
    results = r

#---------------------- C 파트 ------------------------
loop = asyncio.get_event_loop()
loop.run_until_complete(main())
loop.close
e = time.time()

print("{0:.2f}초 걸렸습니다".format(e - s))

#결과
#0.88초 걸렸습니다
```

필자는 0.88초 걸렸다. 만약 비동기식 프로그래밍이 아니었다면 수 초 걸렸을 것이다. 상황마다 다르지만 언제나 Blocking 방식 크롤링보다 훨씬 더 빠르다. 코드를 살펴보자. 코드는 C 파트가 가장 먼저 실행되고 B 파트가 다음 실행된다. 그 후 A 파트가 실행된다.

C 파트부터 살펴보자.

```
#---------------------- C 파트 ------------------------
loop = asyncio.get_event_loop()
loop.run_until_complete(main())
loop.close
```

비동기식은 기다림이 없다. 막히면 다음을 실행한다. 그래서 계속 루프가 돌아야 한다. 그런 루프는 만드는 과정이다. loop 객체를 하나 만들고 main 코루틴이 완료될 때까지 계속 루프가 돌게 된다.

B 파트를 살펴보자.

```
#---------------------- B 파트 ------------------------
async def main():
    urls = ["https://wp.me/p9x2W1-x",
            "https://wp.me/p9x2W1-w",
            "https://wp.me/p9x2W1-t",
            "https://wp.me/p9x2W1-q",
            "https://wp.me/p9x2W1-p",
            "https://wp.me/p9x2W1-j",
            "https://wp.me/p9x2W1-h"]

    fts = [asyncio.ensure_future(getpage(u)) for u in urls]
    r = await asyncio.gather(*fts)
    global results
    results = r
```

async에선 함수를 만들 때 앞에 async를 붙여줘야 한다. 그래야 이 함수가 코루틴이 된다. 비동기식 프로그래밍에선 함수가 아닌 '코루틴이라 부른다.

urls는 그냥 크롤링할 페이지들이다.

ftp 변수가 중요한데 주소들을 future 객체로 만들어 fts에 넣었다. future는 계획서이다. 주소들이 getpage() 코루틴에 의해 실행되는 계획이 리스트로 담기는 것이다.

ftp는 아직 계획서이지 실행되는 것은 아니다. 실행은 그다음 줄에서 된다. await asyncio.gather()는 계획서를 실행하고 그 결과를 r 변수에 담는다.

기호가 사용된 이유는 리스트를 해체해서 요소를 전달해야 하기 때문이다.
A 파트를 살펴보자.

```
#---------------------- A 파트 ------------------------
async def getpage(url):
    req = await loop.run_in_executor(None, requests.get,
                                                        url)
    html = req.text
    soup = await loop.run_in_executor(None, BeautifulSoup,
                                html, 'lxml')
    return soup
```

실제 페이지를 불러오는 코루틴이다.

코루틴 안 명령들을 살펴보자. 가장 낯선 명령은 await loop.run_in_executor()이다. 일단 await는 코루틴을 실행하는 예약어이다. 이것을 반드시 사용해야 다른 코루틴을 실행할 수 있다. 실행한 코루틴은 loop.run_in_executor()인데 이는 아주 특별한 코루틴이다.

원래 파이썬에 있는 모든 함수가 비동기식으로 작동될 수 있는 것은 아니다. 코루틴으로 만들어지지 않았기 때문이다. 따라서 지원하는 함수만 코루틴으로써 실행할 수 있다. 하지만 실망할 것은 없다. 코루틴이 아닌 일반 함수를 코루틴처럼 실행할 수 있기 때문이다. 이때 사용하는 코루틴이 loop.run_in_executor()이다.

```
await loop.run_in_executor(None, requests.get, url)
```

위 명령은 결국 requests.get(url)인 것이다. loop.run_in_executor()의 첫 번째 인자는 executor인데 일단은 넘어가자.

getpage() 코루틴은 결국 url을 인자로 받아 페이지를 불러온다.

이 방식이 비동기식 프로그래밍 방식이다. 루프를 만들고 계획서를 만들어 실행하면 된다. 그 과정에서 async로 함수가 아닌 코루틴으로 만들고 await를 사용해 함수 혹은 다른 코루틴을 실행하면 된다.

마지막 편집일시 : 2018년 7월 22일 10:18

03_Jupyter 사용하기
파이썬을 사용하는 방법 중 Jupyter를 사용하는 사람이 많다. 다음과 같이 생겼다.

웹 브라우저 기반으로 작동하며 터미널에서 서버를 구동시켜 사용한다.

Cell을 추가하면서 코딩하는 방식인데 마치 노트북에 필기하면서 수학 문제를 풀 듯 명령어 한 줄 혹은 여러 줄을 입력하기 때문에 Notebook이라 이름 붙었다.

Jupyter Notebook 서버를 구축하기 위해선 Jupyter를 설치해야 한다.

```
user$> pip install jupyter
```

설치 후 다음과 같은 명령으로 서버를 구동한다.

```
user$> jupyter notebook
```

보통 자동으로 웹 브라우저가 실행되면서 접속하는데 만약 그렇지 않다면 화면에 나와 있는 주소를 복사해 브라우저에 접속하면 된다. 접속하면 화면이 나온다.

그냥 탐색기이다. 여기서 jupyter 파일을 선택해 열수도 있고 새로운 jupyter 파일을 만들 수도 있다. 파일을 새로 만들어 보자. 상단 오른쪽에 있는 New 버튼을 누른 후 파이썬3을 선택한다.

새로운 탭이 생기면서 파일이 열린다. 툴바 하단에 박스가 있는데 이것을 Cell이라 한다. 여기에 코드를 넣기도 하고 설명 혹은 제목을 넣기도 한다.

우선 간단한 코드를 넣어보자.

Cell을 선택한 상태에서 B를 누르면 새로운 Cell이 하단에 추가되고 A를 누르면 상단에 Cell이 추가된다.

새로운 Cell을 하단에 추가해 다음 코드도 입력한다.

Cell을 실행해 보자. 해당 Cell을 선택한 후 단축키로 Shift+Enter 누르면 된다. 현재 두 개 Cell을 모두 실행하면 출력이 필요한 경우 결과가 화면에 바로 나타난다. 이 출력은 변하지 않고 해당 Cell의 결과로써 자리 잡게 된다. 이번엔 Cell에 일반 텍스트를 입력해보자. 아래 Cell을 선택하고 키보드에서 M을 선택해보자. 그럼 Cell의 형태가 바뀌는데 Code에서 Markdown으로 바뀌게 된 것이다. Markdown은 텍스트를 입력할 수 있는 Cell이다. 다시 Code Cell로 바꾸고 싶다면 Y를 누르면 된다.

이 Cell에 텍스트를 입력하고 마찬가지로 Shift+Enter를 실행해 보자.

위 화면처럼 내용이 화면에 그대로 입력된 상태가 된다. 코딩하면서 넣고 싶은 텍스트를 넣으면 된다. IDE에선 모두 주석으로 처리하겠지만 Jupyter Notebook에선 워드처럼 작성 할 수 있다. 수정하고 싶다면 더블클릭하면 된다.

제목도 넣을 수 있다. Markdown Cell에 #기호를 넣고 텍스트를 입력하기만 하면 된다.

Shift+Enter키를 눌러 실행한다.

"#는 1차 제목이고 ##는 2차 제목이다. 이렇게 #기호를 늘리면 된다."

이렇게 Notebook에 필기하듯 코드와 내용, 제목을 넣으면서 코딩하면 된다. 파이썬은 그래프를 그리는 경우도 많고 인터프리터 언어라 과정의 결과를 확인하면서 코딩하는 경우가 많으므로 Jupyter가 유용할 때가 많다.

마지막 편집일시 : 2018년 7월 22일 10:18

04_데이터 분석을 위한 3종 패키지(numpy, pandas, matplotlib)
데이터 분석을 도와주는 패키지인 numpy, pandas, matplotlib에 대해서 알아보자. 굳이 데이터 분석가가 아니더라도 개발자는 데이터를 다루기 때문에 데이터를 분석하고 다루는 방법을 익혀두면 두고두고 사용할 일이 많다. 이번 장에서 설명하는 세 개의 패키지는 파이썬의 활용을 크게 확장 시켜 줄 것이다.

데이터 분석 시 주로 아래와 같은 테이블을 데이터로 다룬다.

	model	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
0	Mazda RX4	21.0	6	160.0	110	3.90	2.620	16.46	0	1	4	4
1	Mazda RX4 Wag	21.0	6	160.0	110	3.90	2.875	17.02	0	1	4	4
2	Datsun 710	22.8	4	108.0	93	3.85	2.320	18.61	1	1	4	1
3	Hornet 4 Drive	21.4	6	258.0	110	3.08	3.215	19.44	1	0	3	1
4	Hornet Sportabout	18.7	8	360.0	175	3.15	3.440	17.02	0	0	3	2
5	Valiant	18.1	6	225.0	105	2.76	3.460	20.22	1	0	3	1
6	Duster 360	14.3	8	360.0	245	3.21	3.570	15.84	0	0	3	4
7	Merc 240D	24.4	4	146.7	62	3.69	3.190	20.00	1	0	4	2
8	Merc 230	22.8	4	140.8	95	3.92	3.150	22.90	1	0	4	2
9	Merc 280	19.2	6	167.6	123	3.92	3.440	18.30	1	0	4	4
10	Merc 280C	17.8	6	167.6	123	3.92	3.440	18.90	1	0	4	4
11	Merc 450SE	16.4	8	275.8	180	3.07	4.070	17.40	0	0	3	3
12	Merc 450SL	17.3	8	275.8	180	3.07	3.730	17.60	0	0	3	3

위 데이터는 R 언어에 내장된 mtcars라는 데이터 샘플이다. 이 데이터는 자동차들에 대한 여러 값을 가지고 있다.

hp는 마력이고 wt는 자동차의 무게를 의미한다. 그리고 cyl는 실린더 수를 나타낸다.

위 데이터는 테이블 형태인데 주로 데이터 분석 시 사용하는 데이터 형태이다. 이런 형태를 DB에선 '관계형 데이터베이스'라 부르기도 한다. 이런 데이터는 주로 행과 열로 나누는데 열을 통계에선 변수라 부르고 행을 관측치라 부른다.

파이썬에서 위와 같은 데이터를 DataFrame이라 부른다. DataFrame을 만들고, 다루기 위해서는 pandas라는 패키지가 필요하다. 먼저 설치부터 해보자.

```
>>> pip install pandas
>>> pip install numpy
>>> pip install matplotlib
```

pandas 외 다른 패키지도 함께 설치했다. numpy는 벡터형 데이터와 matrix를 다루기 위해 사용하고 matlibplot은 데이터를 시각화하기 위해 사용한다.

데이터 분석하기

```
a=1
b='hello'
```

위와 같이 값 하나를 가진 변수를 '스칼라'라고 부른고

```
a = [1, 2, 3]
b = ['hello', 'world']
```

위와 같이 여러 값을 가진 변수를 '벡터'라 부른다. 데이터 분석은 주로 이런 '벡터를 다룬다.

DataFrame의 변수도 결국 '벡터'이다. 이런 '벡터'를 pandas에선 Series라 부르고 numpy는 ndarray라 부른다. 이들에 대해선 잠시 뒤 자세히 살펴볼 것이다. 우선 파이썬에서 제공하는 벡터를 다루는 몇 가지 함수를 알아보자.

```
>>> all([1, 1, 1])          #벡터 데이터 모두 True면 True 반환
>>> any([1,0,0])            #한 개라도 True면 True 반환
>>> max([1,2,3])            #가장 큰 값을 반환한다.
>>> min([1,2,3])            #가장 작은 값을 반환한다.
>>> list(range(10))         #0부터 10까지 순열을 만듦
>>> list(range(3,6))        #3부터 5까지 순열을 만듦 ? 6?
>>> list(range(1, 6, 2))    #1부터 6까지 2단위로 순열을 만듦
```

벡터 요소에 대해 검사하고 결과값을 반환하는 함수들이다. 데이터 분석을 위해선 위 함수 외 평균, 분산 등을 구할 수 있는 함수가 필요하다. 이런 함수들은 numpy, pandas가 제공한다.

먼저 아래 주소로 이동해 Sample로 사용할 Data를 내려받는다.
https://goo.gl/iJZSNh

이 파트는 가능하다면 Jupyter Notebook에서 진행하길 바란다. 많은 그래프와 데이터를 확인하기 때문에 Jupyter Notebook이 편리하다. Jupyter를 사용하면 데이터 분석을 맛보는 시간이 될 것이다.

내려받은 파일은 Jupyter 파일과 함께 둔다.
---

pandas
패키지를 import 한다.

```
In [      ] :   import pandas as pd
```

많은 사람이 pandas의 네이스페이스로 pd를 사용한다. 다른 이름을 사용해도 좋다. csv파일을 불러와야 한다.

```
In [      ] :   df = pd.read_csv("mtcars.csv")
```

데이터의 구조를 먼저 살펴보자. 먼저 관측치 5개만 출력해서 살펴보자.

```
In [      ] :   df.head()
```

	model	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
0	Mazda RX4	21.0	6	160.0	110	3.90	2.620	16.46	0	1	4	4
1	Mazda RX4 Wag	21.0	6	160.0	110	3.90	2.875	17.02	0	1	4	4
2	Datsun 710	22.8	4	108.0	93	3.85	2.320	18.61	1	1	4	1
3	Hornet 4 Drive	21.4	6	258.0	110	3.08	3.215	19.44	1	0	3	1
4	Hornet Sportabout	18.7	8	360.0	175	3.15	3.440	17.02	0	0	3	2

head() 함수는 데이터의 맨 앞부분 5개를 보여준다. 반대로 df.tail()는 데이터의 뒷부분 5개를 보여준다.

```
In [      ] :   df.tail()
```

	model	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
27	Lotus Europa	30.4	4	95.1	113	3.77	1.513	16.9	1	1	5	2
28	Ford Pantera L	15.8	8	351.0	264	4.22	3.170	14.5	0	1	5	4
29	Ferrari Dino	19.7	6	145.0	175	3.62	2.770	15.5	0	1	5	6
30	Maserati Bora	15.0	8	301.0	335	3.54	3.570	14.6	0	1	5	8
31	Volvo 142E	21.4	4	121.0	109	4.11	2.780	18.6	1	1	4	2

만약 특정 범위의 관측치만 보고 싶다면 행에 대해 슬라이싱 하면 된다.

In [ ] : df[0:2]

	model	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
0	Mazda RX4	21.0	6	160.0	110	3.9	2.620	16.46	0	1	4	4
1	Mazda RX4 Wag	21.0	6	160.0	110	3.9	2.875	17.02	0	1	4	4

변수명만 따로 살펴보자.

In [ ] : df.columns
out [ ] : Index(['model', 'mpg', 'cyl', 'disp', 'hp', 'drat', 'wt', 'qsec', 'vs', 'am', 'gear', 'carb'], dtype='object')
columns라는 속성에 모든 변수가 저장되어 있다. 이제 데이터의 여러 통계 정보를 살펴보자. 간략하게 요약해서 알려주는 함수가 있다.

In [ ] : df.describe()

	mpg	cyl	disp	hp	drat	wt
count	32.000000	32.000000	32.000000	32.000000	32.000000	32.000000
mean	20.090625	6.187500	230.721875	146.687500	3.596563	3.217250
std	6.026948	1.785922	123.938694	68.562868	0.534679	0.978457
min	10.400000	4.000000	71.100000	52.000000	2.760000	1.513000
25%	15.425000	4.000000	120.825000	96.500000	3.080000	2.581250
50%	19.200000	6.000000	196.300000	123.000000	3.695000	3.325000
75%	22.800000	8.000000	326.000000	180.000000	3.920000	3.610000
max	33.900000	8.000000	472.000000	335.000000	4.930000	5.424000

	qsec	vs	am	gear	carb
count	32.000000	32.000000	32.000000	32.000000	32.0000
mean	17.848750	0.437500	0.406250	3.687500	2.8125
std	1.786943	0.504016	0.498991	0.737804	1.6152
min	14.500000	0.000000	0.000000	3.000000	1.0000
25%	16.892500	0.000000	0.000000	3.000000	2.0000
50%	17.710000	0.000000	0.000000	4.000000	2.0000
75%	18.900000	1.000000	1.000000	4.000000	4.0000
max	22.900000	1.000000	1.000000	5.000000	8.0000

각 변수에 해당하는 통계적 의미인 count(관측치 값), mean(평균), std(표준편차), min(최솟값), max(최댓값)을 알려준다. 25%, 50%, 75%는 데이터 4등분하여 25%에 해당하는 값과 50%, 75%에 해당하는 값을 나타낸다. 물론 평균, 표준편차 등을 알려주는 함수는 별도로도 있다.

평균을 구하려면

In [ ] : df.mean()
out [ ] : mpg 20.090625
cyl 6.187500
disp 230.721875
hp 146.687500
drat 3.596563
wt 3.217250
qsec 17.848750
vs 0.437500
am 0.406250
gear 3.687500
carb 2.812500
dtype: float64

변수별로 평균만 나타난 것을 확인 할 수 있다. pandas에서는 특정 변수를 기준으로 그룹을 지어 통계값을 구할 수도 있다. 예를 들어 "cyl 변수별로 hp 평균 값"을 구하는 것이다.

In [ ] : df.groupby(['cyl']).mean()

cyl mpg disp hp drat wt qsec
4 26.663636 105.136364 82.636364 4.070909 2.285727 19.137273
6 19.742857 183.314286 122.285714 3.585714 3.117143 17.977143
8 15.100000 353.100000 209.214286 3.229286 3.999214 16.772143
cyl vs am gear carb
4 0.909091 0.727273 4.090909 1.545455
6 0.571429 0.428571 3.857143 3.428571
8 0.000000 0.142857 3.285714 3.500000
cyl는 실린더 수인데 4개인 경우에 해당하는 평균과 6개, 8개인 경우에 해당하는 평균을 각각 구해 알려준다.

hp 변수에 대해서만 실린더 수 별로 평균값을 구하고 싶다면 다음과 같이 한다.
In [ ] : df.groupby(['cyl'])['hp'].mean()
out [ ] : cyl
4 82.636364
6 122.285714
8 209.214286
Name: hp, dtype: float64
결국, 위 코드는 "실린더 수별로 마력의 평균"을 구한 것이다.

이처럼 그룹을 지을 수도 있고 변수를 선택할 수도 있다. 만약 단순히 특정 변수를 선택해 평균을 구하고 싶다면 다음과 같이 한다.

In [ ] : df['hp'].mean()
out [ ] : 146.6875
변수를 선택하는 여러 방법이 있다.

In [ ] : df.hp
out [ ] : df['hp']
두 방법의 결과는 같다. 편한 방법을 사용하면 된다. 변수 안에서 슬라이싱을 할 수 있다.

In [ ] : df.hp[30]
out [ ] : 335

여러 값을 슬라이싱 할 수도 있다.

```
 In [      ] :   df.hp[0:3]
out [      ] :   0     110
         1     110
         2      93
```

변수만 선택했을 때 자료형은 무엇인지 살펴보자.

```
 In [      ] :    type(df.hp)
out [      ] :   <class 'pandas.core.series.Series'>
```

Series라고 알려준다. 통계적 의미에선 변수이지만 pandas는 Series라 부른다. Pandas의 DataFrame은 이 Series의 모음인 것이다.

지금부터는 hp 변수만 따로 저장해 Series에 대해서 자세히 살펴보자.

```
 In [      ] :   s = df.hp
In [      ] :   s
out [      ] :   0   110
         1   110
         2   93
         3   110
         4   175
         5   105
         6   245
         7   62
         ...생략합니다....
```

Series는 크게 index와 value로 나뉜다. 왼쪽이 index이고 오른쪽이 value이다. index는 아무 조치를 하지 않는다면 0부터 시작한다. mtcars 데이터를 잘 보면 models라는 변수가 있는데 이 값은 자동차 이름이다. 이 이름을 index로 넣어보자.

```
 In [      ] :   s.index = df.model
In [      ] :   s
out [      ] :Mazda RX4   110
         Mazda RX4 Wag    110
         Datsun 710   93
         Hornet 4 Drive   110
         Hornet Sportabout 175
         Valiant 105
         Duster 360   245
```

이제 이름으로 선택할 수 있다.

```
 In [      ] :   s['Datsun 710']
out [      ] :   93
```

: 기호도 사용할 수 있다.

```
 In [      ] :   s['Datsun 710':'Valiant']
out [      ] :   model
         Datsun 710   93
         Hornet 4 Drive   110
         Hornet Sportabout    175
         Valiant 105
         Name: hp, dtype: int64
```

이렇게 만들어진 Series는 연산도 가능하다.

```
 In [      ] :   print(s*2)
out [      ] :   model
        Mazda RX4    220
        Mazda RX4 Wag    220
        Datsun 710   186
        Hornet 4 Drive   220
        Hornet Sportabout    350
        Valiant 210
        Duster 360   490
        ...생략합니다...
```

pandas의 series의 통계 값을 구하는 함수를 정리하면 다음과 같다.

```
In [      ] :   df.hp.mean()    #평균
In [      ] :   df.hp.std()     #표준편차
In [      ] :   df.hp.var()     #분산
```

==========

==========

numpy

Series는 벡터이다. 이 벡터를 pandas에선 series라 부르지만 numpy에선 ndarray라 부른다. 두 패키지 모두 벡터를 다루지만 사용 용도는 조금 다르다. 그럼 numpy에 대해 알아보자.

```
In [      ] :   import numpy as np
In [      ] :   np.mean(df.hp)          #numpy로 평균 구하기
In [      ] :   np.std(df.hp)           #numpy로 표준편차 구하기
In [      ] :   np.var(df.hp)           #numpy로 분산 구하기
```

pandas의 series는 numpy에서 모두 인식된다. 벡터는 다음과 같이 일차원이다.

{0,1,2,3,4,5,6,7}

다음은 이차원 메트릭스이다.

([0,1,2,3], [4,5,6,7])

numpy는 다차원 구조를 가진 데이터를 처리하기 위한 용도로 만들어졌다. 삼차원 혹은 그 이상의 다차원까지도 처리한다. DataFrame과 Matirx는 비슷해 보이지만 차이점이 있다. Matrix는 수치를 처리하는 반면 DataFrame은 '문자열', '수치' 등 다양한 데이터형을 다룬다. Matrix는 행이든 열이든 모두 의미가 있을 수 있지만 DataFrame은 주로 열에 대해서 의미가 있다. 이제 한번 만들어 보도록 하자. 다음과 같이 일차원 벡터를 만들 수 있다.

```
 In [      ] :   a = np.array(list(range(8)))
                 a
out [      ] :   array([0, 1, 2, 3, 4, 5, 6, 7])
```

이를 이차원의 메트릭스로 만들려면 단순히 shape 값만 바꿔 주면 된다.

```
 In [      ] :   a.reshape((2, 4))
out [      ] :   array([[0, 1, 2, 3],
                       [4, 5, 6, 7]])
```

2x4 메트릭스가 만들어졌다. 행이 2이고 열이 4이다. '행렬'이라 부르니까 앞 숫자가 '행'이다. 이처럼 (2,4)를 numpy에선 shape라 부른다. 그래서 reshape() 함수를 이용하면 행렬의 구조를 바꿀 수 있다.

```
In [ ] : a.reshape((4,2))
out [ ] : array([[0, 1],
                 [2, 3],
                 [4, 5],
                 [6, 7]])
```

삼차원으로 바꿀 수도 있다.

```
In [ ] : a.reshape((2,2,2))
out [ ] : array([[[0, 1],
                  [2, 3]],
                 [[4, 5],
                  [6, 7]]])
```

shape의 값을 바꿔 주는 것만으로 메트릭스의 구조를 쉽게 바꿀 수 있다. 이런 유연함이 ndarray형의 특징이다. 이렇게 벡터를 만들고 shape의 구조를 만들어 사용하는 때도 많고 또는 처음부터 구조를 만들어 사용하는 때도 많다. 다음과 같이 0으로 채워진 메트릭스를 만들 수 있다.

```
In [ ] : np.zeros((2, 4))
out [ ] : array([[ 0.,  0.,  0.,  0.],
                 [ 0.,  0.,  0.,  0.]])
```

0이 아닌 다른 숫자로도 만들 수 있다.

```
In [ ] : np.full((2, 4), 7)
out [ ] : array([[7, 7, 7, 7],
                 [7, 7, 7, 7]])
```

대각선으로만 1인 메트릭스를 만들 수도 있다.

```
In [ ] : np.eye(3)
out [ ] : array([[ 1.,  0.,  0.],
                 [ 0.,  1.,  0.],
                 [ 0.,  0.,  1.]])
```

직접 한 번에 만들 수도 있다.

```
In [ ] : a = np.array([[0,1,2], [3,4,5]])
In [ ] : a
out [ ] : array([[0, 1, 2],
                 [3, 4, 5]])
```

numpy는 메트릭스를 다루기 때문에 메트릭스 연산을 할 수 있다. 벡터(ndarray)를 먼저 만들자.

```
In [ ] : a = np.array(list(range(1,10)))
         a
out [ ] : array([1, 2, 3, 4, 5, 6, 7, 8, 9])
In [ ] : b = np.array(list(range(11,20)))
         b
out [ ] : array([11, 12, 13, 14, 15, 16, 17, 18, 19])
```

shape 재정의해 Matrix를 만든다.

```
In [ ] : a = a.reshape(3, 3)
         b = b.reshape(3, 3)
         a
         b
out [ ] : array([[1, 2, 3],
```

```
                    [4, 5, 6],
                    [7, 8, 9]])
        array([[11, 12, 13],
                   [14, 15, 16],
                   [17, 18, 19]])
```

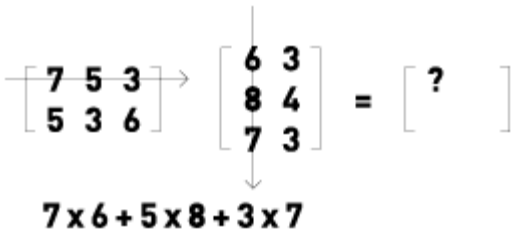
이제 두 메트릭스를 연산해보자.

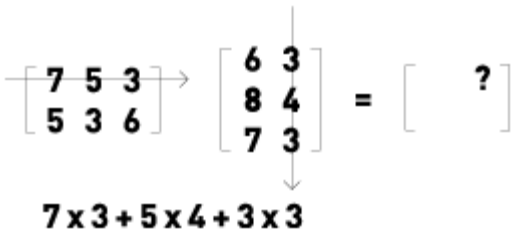
```
 In [      ] :   a + b
out [      ] :   array([[12, 14, 16],
                   [18, 20, 22],
                   [24, 26, 28]])
```

메트릭스의 요소끼리 서로 더해진 것을 확인할 수 있다. 빼기, 곱하기, 나누기 모두 가능하다.

```
 In [      ] :   a - b
out [      ] :   array([[-10, -10, -10],
                   [-10, -10, -10],
                   [-10, -10, -10]])
In [      ] :   a * b
out [      ] :   array([[ 11,  24,  39],
                   [ 56,  75,  96],
                   [119, 144, 171]])
In [      ] :   a / b
out [      ] :   array([[ 0.09090909,  0.16666667,  0.23076923],
                   [ 0.28571429,  0.33333333,  0.375       ],
                   [ 0.41176471,  0.44444444,  0.47368421]])
```

곱하기 경우 요소끼리 곱하는 경우 외 행렬곱이라는 계산 방법이 있다. 아래 그림과 같이 연산하는 것이다.





앞서 만든 matrix a, b에 적용해 보자.

```
 In [      ] :   a.dot(b)
out [      ] :   array([[ 90,  96, 102],
                   [216, 231, 246],
                   [342, 366, 390]])
```

행렬곱으로 연산 되었다. numpy에는 np.matrix() 함수가 있다 matrix를 만드는 함수인데 이 함수로 matrix를 만들면 우리가 지금까지 살펴본 데이터형과 다른 데이터형이 만들어진다. numpy에서는 ndarray와 matrix 데이터형 두 가지가 있으며 서로 사용방법이 약간 다르다.

이 부분이 헷갈릴 수 있어 미리 언급한다. 두 차이를 이해하는 것도 좋지만 입문단계에선 헷갈릴 뿐이다. ndarray에 충분히 익숙해지고 matrix를 살펴보는 것이 좋을 것 같다.

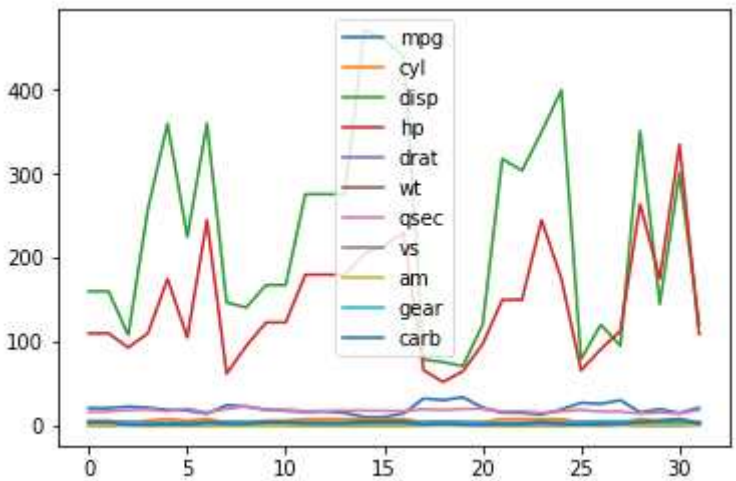
시각화

python에서 시각화는 주로 matplolib 패키지를 사용한다. 더욱이 이 패키지는 pandas와 조화를 이루면서 강력한 기능을 발휘한다. 바로 예제를 통해 살펴보자.

```
In [      ] :   import pandas as pd
        import numpy as np
        import matplotlib.pyplot as plt
        %matplotlib inline    #그래프를 브라우저에 나타내기 위한 코드
```

다음과 같이 데이터를 불러온다.

```
In [      ] :   df = pd.read_csv("mtcars.csv")
        df.plot()
        plt.show()
```

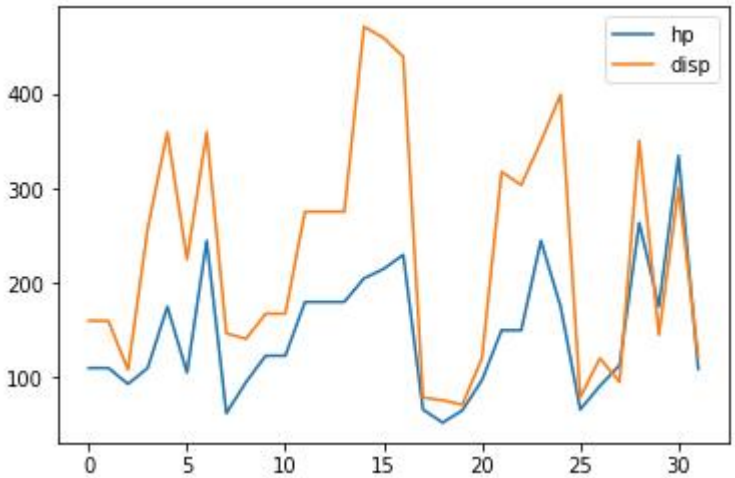


단순히 DataFrame객체를 만들고 그 객체의 메서드인 plot() 함수를 실행하는 것만으로 그래프가 그려진다. DataFrame객체에는 matplotlib를 이용해 그래프를 그리는 메서드가 이미 있다. 그러니 matplotlib 객체를 따로 만들지 않아도 된다. 그리고 코드 중 plt.show() 함수는 그래프를 보여주는 함수인데 Pycharm을 사용한다면 반드시 넣어줘야 하고 Jupyter를 사용하고 있다면 굳이 사용하지 않아도 된다. 위 그래프는 df의 모든 변수가 그래프로 나타났다. 이 중에서 원하는 한 개의 변수만 나타내 보자.

```
In [13] :   df2 = df.hp #hp 변수를 df2에 저장
     df2.plot()
```

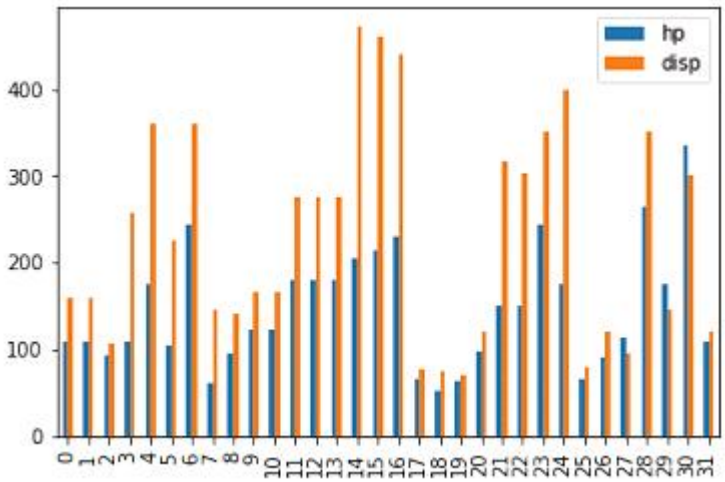
단순하게 DataFrame의 변수 하나만 따로 저장한 후 plot() 함수를 실행했다. 물론 따로 저장하지 않고 df.hp.plot()를 실행해도 무방하다. 명령어 중 plt.figure() 함수는 새로운 그래프를 그리겠다는 명령어이다. 이 명령어가 없다면 기존 그래프에 겹쳐서 그려지게 된다. 이 또한 Jupyter를 사용한다면 plt.figure()와 plt.show()는 생략해도 된다. Pycharm을 사용한다면 반드시 넣어야 한다. 이제 여러 개 변수를 그려보자.

```
In [      ]: df2 = df[['hp','disp']]
     df2.plot()
```



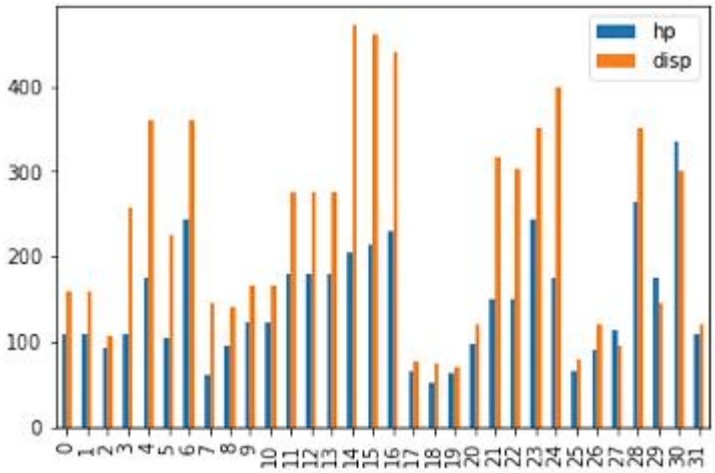
이 그래프를 막대 그래프로 나타내 보자. 인자 하나만 추가하면 된다.

In [ ]: df2.plot(kind='bar')



kind인자에 어떤 그래프를 그리고 싶은지 명시하기만 하면 된다. 근데 이런 그래프는 다른 방법으로도 그릴 수 있다.

In [ ]: df2.plot.bar()

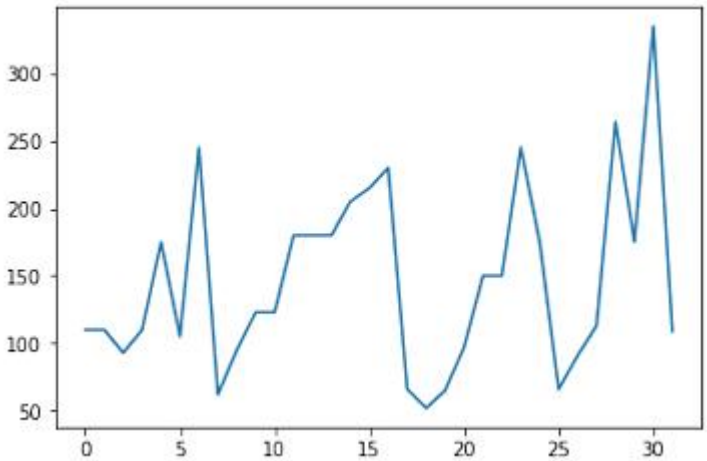


조금 전 그래프와 같다. 즉, 아래 두 코드는 같다.

```
In [     ]: df2.plot(kind='bar')
    df2.plot.bar()
```

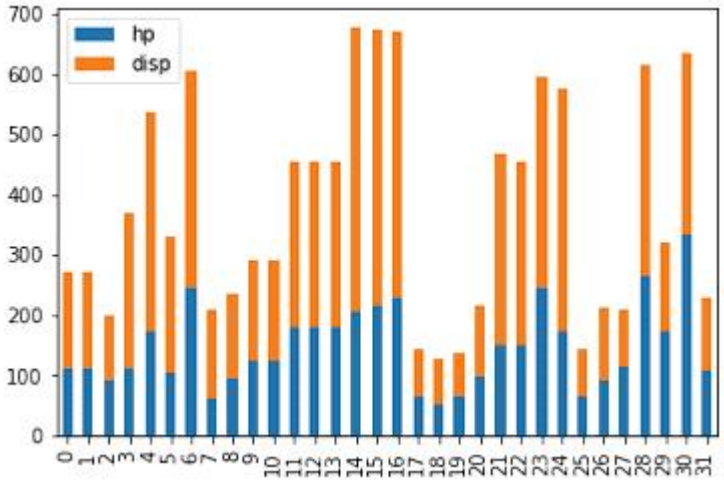
참고로 df2.plot.을 입력 후 "Tab"키를 누르면 사용 가능한 그래프 목록이 나온다. 이때 목록에서 선택하면 된다. 근데 pandas의 DataFrame객체 메서드가 아닌 matplotlib의 함수를 바로 실행해 그래프를 그릴 순 없을까. 당연히 가능하다.

```
In [     ]: plt.plot(np.arange(len(df.hp)), df.hp)
```



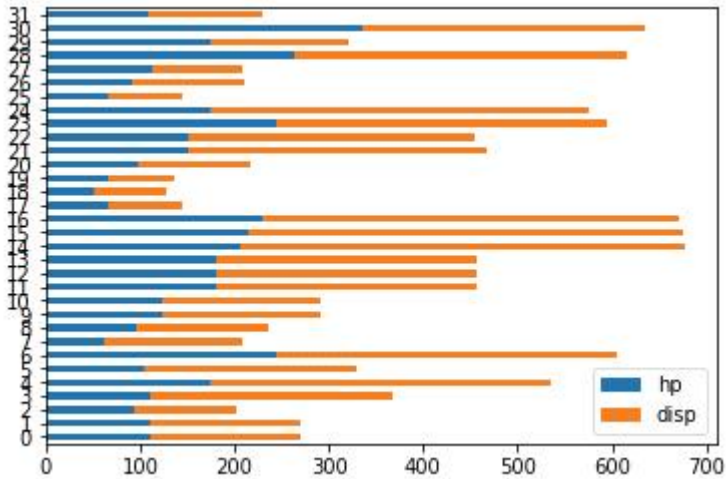
plt 네임스페이스를 통해 관련된 함수를 실행했다. 결국, 다음 두 가지 방법으로 matplotlib를 사용할 수 있는 것이다. 객체.plot() plt.plot(객체 변수) 막대 그래프에는 몇 가지 옵션이 있다. 먼저 누적된 막대 그래프를 그려보자.

```
In [     ]: df2.plot.bar(stacked=True)
```



가로 모드로 막대 그래프를 그리기 위해선 barh()를 사용한다.

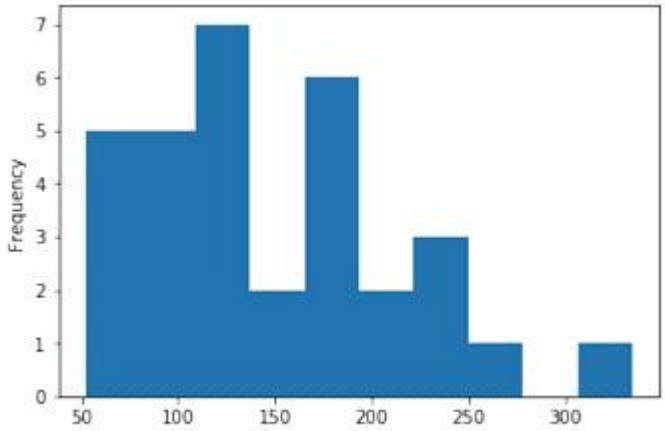
```
In [      ]: df2.plot.barh(stacked=True)
```



히스토그램 그리기

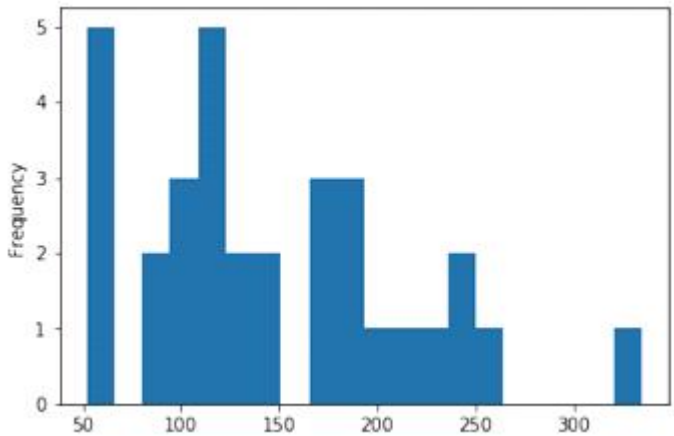
히스토그램도 간단히 그릴 수 있다.

```
In [      ]: df1 = df.hp
    df1.plot.hist()
```



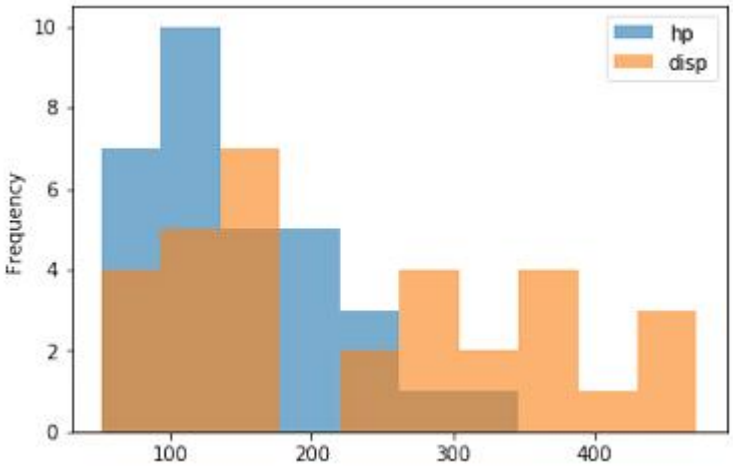
한 변수에 대해 히스토그램을 그리면 y축은 자연스럽게 빈도수가 된다. x축은 알맞게 구간이 설정되고 그 안에 속한 데이터 값을 카운팅 한다. 만약 간격을 조정하고 싶다면 bins 인자를 추가하면 된다.

```
In [      ]: df1.plot.hist(bins=20)
```



만약 두 변수가 DataFrame에 있다면 함께 나타난다. 이때 겹치는 부분이 있으니 투명도를 조절하면 좋다.

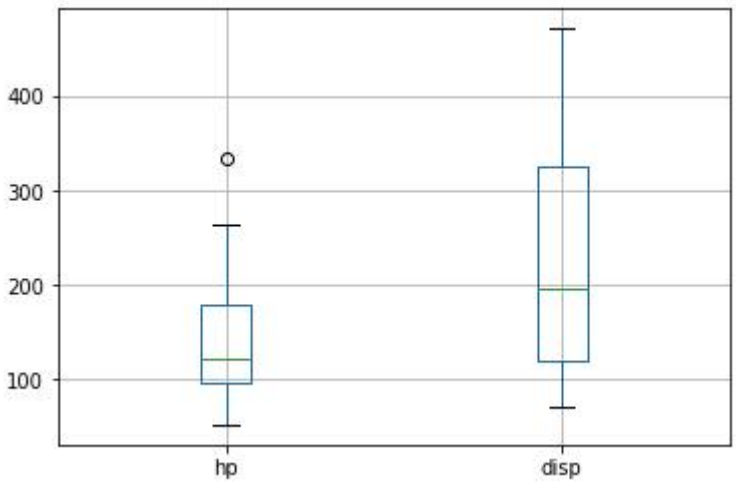
```
In [23]:    df2.plot.hist(alpha=0.6)
```

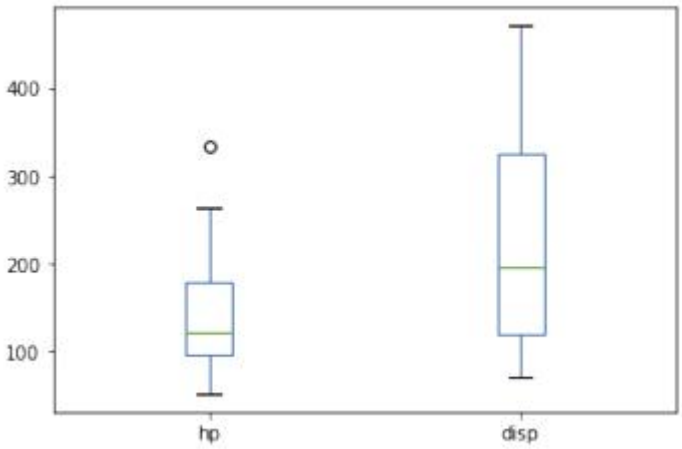


Box Plot 그리기

Box Plot은 중앙값, IQR, 아웃라이어 등 데이터의 특징을 모두 나타낸다. Box Plot은 두 가지 방법으로 그릴 수도 있다.

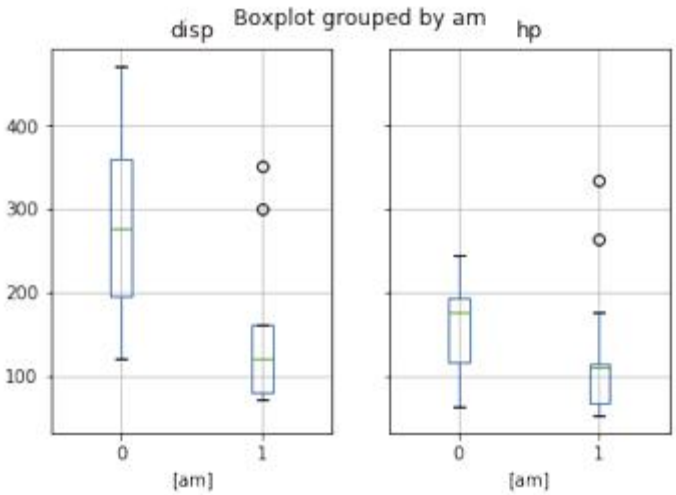
```
In [      ]: df2.boxplot()
    df2.plot.box()
```





차이점은 크지 않으나 boxplot() 함수는 그룹을 지어 그릴 수 있도록 지원한다.

```
In [      ]:  df3 = df[['hp', 'disp', 'am']]
      df3.boxplot(by='am')
```

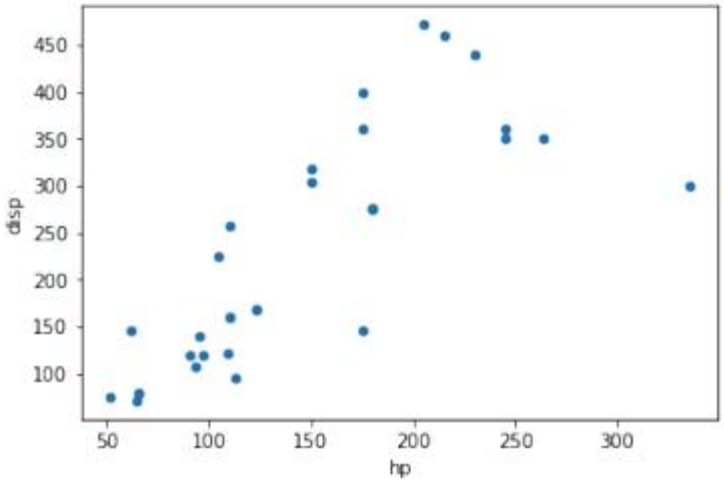


am 변수는 자동기어 여부를 나타내는 변수로써 0 혹은 1이다. 이 두 경우에 대해 각각을 그래프에서 나타내 준다.

산점도 그리기

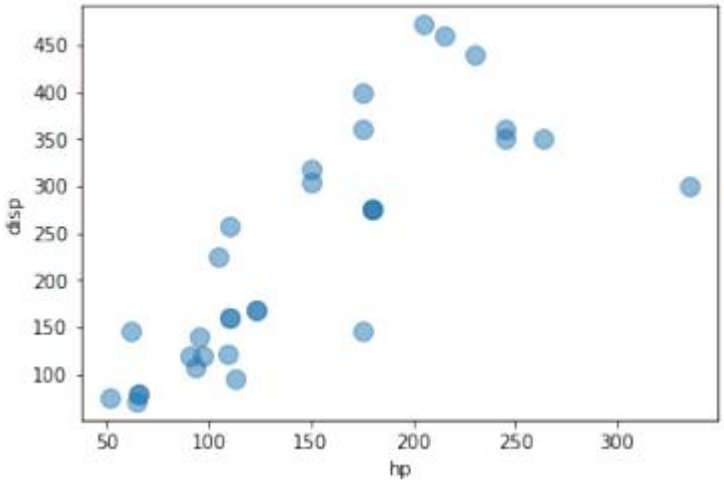
산점도는 두 변수 간의 상관관계를 보기 위해 그리는데 그리는 방법은 다음과 같다.

In [26]: df3.plot.scatter(x='hp', y='disp')



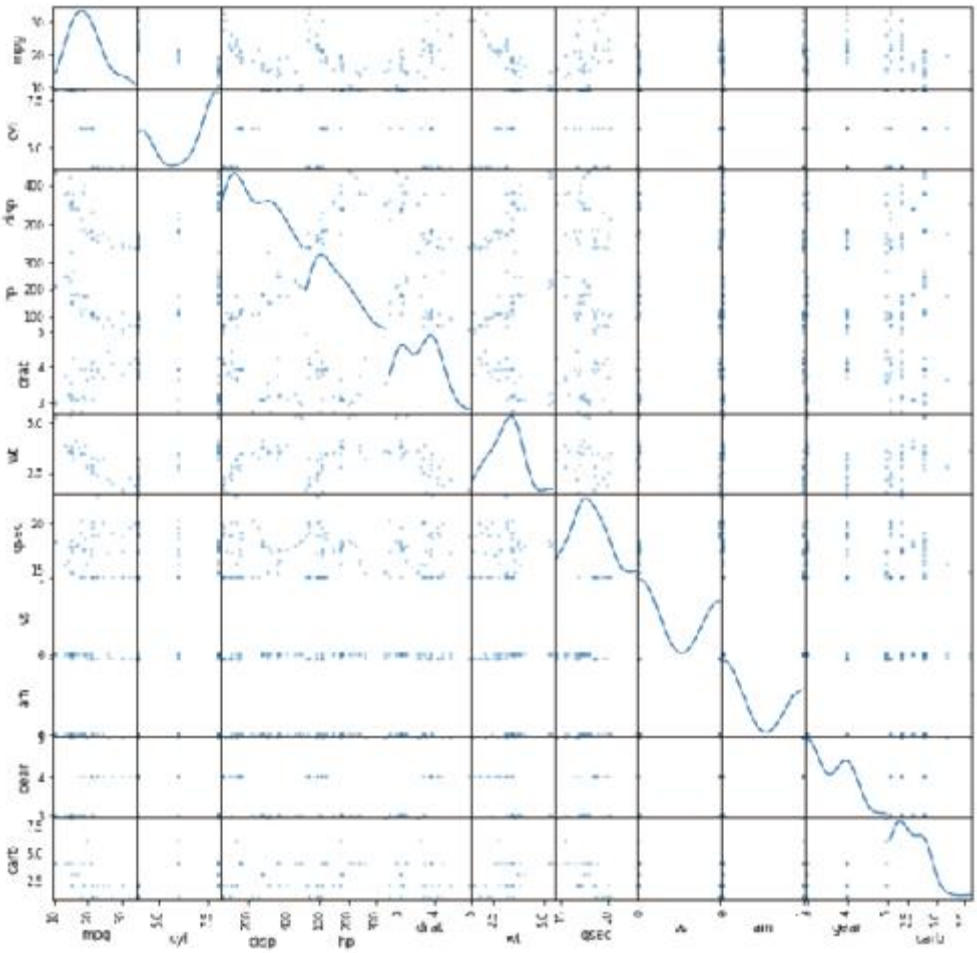
원의 크기와 투명도를 조절할 수 있다.

In [ ]: df3.plot.scatter(x='hp', y='disp', s=100, alpha=0.5)



pandas 패키지에는 DataFrame의 모든 변수에 대한 산점도를 한 번에 보여주는 함수가 있어 편리하다.

```
In [     ]:  from pandas.plotting import scatter_matrix
     scatter_matrix(df, alpha=0.4, figsize=(12, 12),
     diagonal='kde')
```



모든 변수를 X축과 Y축에 나열되고 서로 만나는 지점에 두 변수 간의 산점도를 보여주고 있다. 대각선 부분은 X축 Y축 모두 자기 자신이기 때문에 자신의 값을 그대로 보여준다.

마지막 편집일시 : 2018년 7월 22일 10:18
==========